OPEN SOURCE | LINUX | AUTOMATE | SCALE

The Complete Linux Engineering Handbook

Foundations to DevOps Internals - 28 Modules

$Same Kernel.
More Possibilities.
A Better Tomorrow.

```
$ uname -a
Linux 6.8.0 x86_64

$ whoami
linux_enthusiast

$ cat /etc/os-release
   NAME="Linux"
   VERSION="2024"
   ID=linux
   ID_LIKE=unix

$ systemctl status
```

COMMAND LINE MASTERY | SYSTEM ADMINISTRATION | ADVANCED LINUX | LINUX FOR DEVOPS

- REAL COMMANDS
- PRACTICAL EXAMPLES
- DIAGRAMS & FLOWS
- TROUBLESHOOTING GUIDES
- INTERVIEW PREP
- AND MORE...



LINUX POWERS THE MODERN WORLD

EXPLORE
LEARN
BUILD
GROW

COMMUNITY | KNOWLEDGE | BETTER SYSTEMS

Prepared by
Shaam Natheshwar
DevOps Engineer

OCTOBER 2026

Table of Contents

How to Use This Handbook 8

PART 1: FOUNDATIONS 9

Module 1: Linux Fundamentals 9

What is Linux 9

Linux History 9

Linux Kernel vs Linux Distribution 9

Popular Linux Distribution Families 10

Choosing the Right Distribution 10

Release Cycles: LTS vs Rolling Release 10

Open Source Licensing 11

Linux vs Unix vs macOS 11

Linux vs Windows — Fundamental Differences 11

Linux Use Cases 11

Linux Standard Base (LSB) & Filesystem Hierarchy Standard (FHS) 12

Module 2: Linux Installation & Boot Process 12

Installation Environments 12

Disk Partitioning 12

BIOS vs UEFI 13

Bootloader — GRUB2 13

Linux Boot Process — Detailed Flow 14

Kernel Parameters at Boot 14

Boot Troubleshooting Modes 14

Module 3: Linux Filesystem Hierarchy 15

The Filesystem Hierarchy Standard (FHS) Map 15

Directory-by-Directory Reference 16

Absolute vs Relative Paths 18

Hard Links vs Symbolic Links 18

Special File Types 19

PART 2: COMMAND LINE MASTERY 20

Module 4: Shell & Terminal Basics 20

What is a Shell 20

Shell Types 20

Shell Prompt Anatomy 20

Command Syntax 21

Getting Help 21

Command History 21

Tab Completion & Keyboard Shortcuts 21

Terminal Multiplexers — tmux 22

Module 5: File & Directory Operations 23

Long Listing Format Interpretation 23

find — the Swiss-army search command 23

locate vs find ... 24
File Metadata & Disk Usage ... 24
Module 6: File Content Operations ... 25
Module 7: Text Processing — grep, sed, awk ... 26
grep — Pattern Searching ... 26
Regular Expressions Cheat Sheet ... 26
sed — Stream Editor ... 27
awk — Pattern Scanning & Reporting ... 27
xargs — Building Commands from Input ... 27
Module 8: I/O Redirection & Pipes ... 28
Standard Streams ... 28
Redirection Operators ... 28
Pipes ... 28
tee — Split Output ... 28
Process Substitution ... 29
Special Devices ... 29
Module 9: Shell Scripting ... 29
Script Basics ... 29
Variables ... 30
Special Variables ... 30
Quoting ... 30
Arithmetic ... 30
String Operations ... 31
Conditionals ... 31
Loops ... 31
Functions ... 32
Arrays ... 32
Input/Output ... 32
Error Handling ... 32
Debugging ... 33
Script Best Practices ... 33
PART 3: SYSTEM ADMINISTRATION ... 34
Module 10: User & Group Management ... 34
User Types ... 34
Key Account Files ... 34
User Management Commands ... 35
Group Management ... 35
su vs sudo ... 35
PAM (Pluggable Authentication Modules) ... 36
Resource Limits & Login Restrictions ... 36
Module 11: File Permissions & Access Control ... 36
Standard Permissions ... 36
umask — Default Permission Mask ... 37
Special Permission Bits ... 37
Access Control Lists (ACLs) ... 37

Extended Attributes & Immutability ... 38
File Capabilities ... 38
Module 12: Process Management ... 39
Viewing Processes ... 39
Process States ... 39
Signals & Killing Processes ... 40
Job Control ... 40
Priority Management ... 40
Inspecting Processes via /proc ... 41
Finding What’s Using a File/Port ... 41
System Call & Library Tracing ... 41
Module 13: Service Management with systemd ... 41
systemd vs SysV init ... 41
systemctl — The Control Command ... 42
systemd Unit Types ... 42
Unit File Locations (priority order) ... 43
Anatomy of a Service Unit File ... 43
systemd Targets (Runlevel Equivalents) ... 44
systemd Timers (cron replacement) ... 44
journald & journalctl ... 44
Boot Performance Analysis ... 45
Other systemd Utilities ... 45
Module 14: Package Management ... 45
RPM-Based Systems (RHEL, CentOS, Rocky, Fedora) ... 45
DEB-Based Systems (Debian, Ubuntu) ... 46
Universal / Sandboxed Package Formats ... 46
Building From Source ... 46
Module 15: Storage & Filesystem Management ... 47
Block Devices ... 47
Partitioning Tools ... 47
Creating Filesystems ... 47
Filesystem Repair ... 47
Mounting ... 48
/etc/fstab — Persistent Mounts ... 48
LVM — Logical Volume Manager ... 48
RAID (mdadm — Software RAID) ... 49
Disk Quotas ... 49
Swap Management ... 50
Storage Performance Tools ... 50
Advanced Filesystems ... 50
Module 16: Networking in Linux ... 50
Interface & Address Management (the `ip` command — modern replacement for ifconfig/route/arp) ... 50
NetworkManager ... 51
DNS Configuration ... 51

Hostname ... 51
Network Troubleshooting Toolkit ... 52
Firewall Management ... 52
Bonding, Bridging, VLANs ... 53
Traffic Control & Namespaces ... 53
Module 17: Linux Security ... 54
sudo Hardening ... 54
SSH Hardening (`/etc/ssh/sshd_config`) ... 54
PAM & Password Policy ... 54
Auditing Files for Risk ... 55
SELinux (Security-Enhanced Linux) — Mandatory Access Control ... 55
AppArmor (Ubuntu/Debian's MAC alternative) ... 55
Linux Audit Framework ... 56
Kernel-Level Security Features ... 56
Security Scanning Tools ... 56
Cryptography Tools ... 56
Module 18: Logging & Monitoring ... 57
Syslog Facilities & Severities ... 57
rsyslog ... 57
journald ... 57
logrotate ... 57
Important Log Files ... 58
System Monitoring Tools ... 58
Performance Analysis Methodology ... 58
Module 19: Scheduling & Automation ... 59
cron ... 59
anacron ... 59
at / batch (one-time scheduling) ... 59
Automation Patterns ... 60
Module 20: Network Services Administration ... 60
SSH Server (OpenSSH) ... 60
Web Servers ... 60
DNS Server (BIND) ... 61
DHCP Server ... 61
NFS (Network File System) ... 61
Samba (Windows File Sharing) ... 62
FTP Server (vsftpd) ... 62
Email (Postfix) ... 62
Time Sync (Chrony — modern default on RHEL 8+) ... 62
Proxy Server (Squid) ... 63
PART 4: ADVANCED LINUX ... 64
Module 21: Kernel Management ... 64
Kernel Versioning ... 64
Kernel Modules ... 64
Kernel Parameters (sysctl) ... 64

Kernel Updates & Crash Dumps ... 65
eBPF (extended Berkeley Packet Filter) ... 65
Module 22: Linux Containers & Virtualization ... 66
Namespaces — The Isolation Primitive ... 66
cgroups (Control Groups) — The Resource-Limiting Primitive ... 67
Overlay Filesystem — How Container Images Are Layered ... 67
KVM (Kernel-based Virtual Machine) ... 67
LXC/LXD — System Containers ... 68
Docker on Linux — Kernel-Level View ... 68
Podman — Daemonless Alternative ... 68
systemd-nspawn ... 68
Module 23: High Availability & Clustering ... 69
Pacemaker + Corosync ... 69
Keepalived (VRRP-based failover) ... 69
HAProxy — Load Balancing ... 69
Distributed Storage: GlusterFS & Ceph ... 70
Module 24: Performance Tuning ... 70
CPU Tuning ... 70
Memory Tuning ... 70
Disk I/O Tuning ... 71
Network Tuning ... 71
Application-Level Tuning ... 71
Benchmarking & Profiling ... 71
Module 25: Linux Hardening & Best Practices ... 72
CIS Benchmark Philosophy ... 72
Core Hardening Checklist ... 72
Kernel Hardening (sysctl) ... 72
Boot & Filesystem Hardening ... 72
Mandatory Access Control & Audit Logging ... 73
File Integrity Monitoring (AIDE) ... 73
Automated Hardening Tools ... 73
PART 5: LINUX FOR DEVOPS ... 74
Module 26: Linux for Containers & Kubernetes ... 74
Namespace Debugging Commands ... 74
Inspecting Container Resource Limits via cgroups ... 74
Container Networking at the Kernel Level ... 74
Overlay Filesystem Inspection ... 74
Container Debugging Without `exec` ... 75
Linux Capabilities in Containers ... 75
Seccomp Profiles ... 75
Kernel Tuning for Kubernetes Nodes ... 75
Kubernetes Node-Level Troubleshooting ... 76
Module 27: Linux Automation for DevOps ... 76
Git on Linux ... 76
Build Tools ... 76

Environment Management 77
Scripting Cloud & APIs 77
Cron/Timer-Driven DevOps Tasks 77
Linux in CI/CD 77
Module 28: Linux Troubleshooting Methodology 78
Structured Troubleshooting Flow 78
First Commands When Something Is Wrong 78
Common Problem Playbooks 78

How to Use This Handbook

This handbook follows a 28-module curriculum spanning five parts: Foundations, Command Line Mastery, System Administration, Advanced Linux, and Linux for DevOps. Every module is explained with:

- **Concept explanations** — what it is and why it matters in real-world/production environments
- **Command-line syntax blocks** — formatted exactly as they appear in a terminal
- **Flow diagrams** — ASCII-style architecture and process-flow diagrams for the concepts that are easiest to understand visually (boot process, troubleshooting methodology, LVM stack, container internals, etc.)
- **Practical notes** — gotchas, defaults, and things that matter on the job (DevOps/SRE/Cloud context)

Read it top to bottom as a course, or use the table of contents as a lookup reference while working.

PART 1: FOUNDATIONS

Module 1: Linux Fundamentals

What is Linux

Linux is a free, open-source, Unix-like **operating system kernel** originally written by **Linus Torvalds** in 1991. Technically, “Linux” refers only to the kernel — the core program that manages CPU scheduling, memory, devices, and processes. What most people call “a Linux system” is actually **GNU/Linux**: the Linux kernel combined with GNU userland tools (shell, compilers, coreutils) and a package of applications, assembled into a *distribution*.

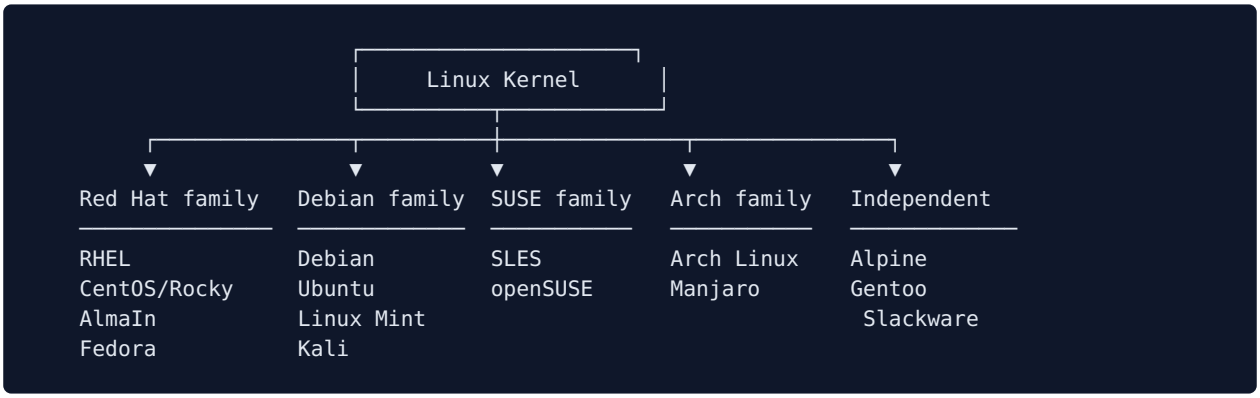
Linux History

- **1983** — Richard Stallman launches the **GNU Project**, aiming to build a complete free Unix-like OS. By the early 1990s GNU had almost everything (compiler, editor, shell) except a working kernel.
- **1991** — Linus Torvalds, a Finnish student, posts to Usenet announcing a “hobby” kernel project, which becomes **Linux**.
- Combining the GNU userland with the Linux kernel produced a fully free operating system — hence **GNU/Linux**.
- Linux is licensed under the **GPLv2**, which is why its source has stayed open and forkable for 30+ years.

Linux Kernel vs Linux Distribution

	Kernel	Distribution (Distro)
Definition	The core program (process/ memory/device management)	Kernel + package manager + userland tools + defaults
Example	`linux-5.15`, `linux-6.6`	Ubuntu, RHEL, Debian, Fedora
Who maintains	kernel.org / Linus Torvalds + maintainers	Vendor or community (Red Hat, Canonical, Debian project)
You install	Rarely directly	Always — this is what you download/boot

Popular Linux Distribution Families



- **Red Hat family** (RHEL, CentOS Stream, Rocky Linux, AlmaLinux, Fedora): RPM packages, `dnf` / `yum`, enterprise-focused, SELinux by default. This is the dominant family in **enterprise DevOps/Cloud jobs** (and the basis of RHCSA/RHCE).
- **Debian family** (Debian, Ubuntu, Linux Mint, Kali): `.deb` packages, `apt`, very common on cloud VMs (AWS EC2 default images, GitHub Actions runners) and desktops.
- **SUSE family** (SLES, openSUSE): RPM-based but uses `zypper`; strong in European enterprise.
- **Arch family** (Arch Linux, Manjaro): rolling release, minimal, `pacman`, popular with power users.
- **Independent** (Alpine — tiny, musl-libc, dominant base image for Docker containers; Gentoo — source-based; Slackware — oldest surviving distro).

Choosing the Right Distribution

- **Enterprise servers / government / banks** → RHEL, Rocky, SLES (support contracts, certification).
- **Cloud-native / containers** → Ubuntu LTS, Alpine, Amazon Linux.
- **Learning for certifications (RHCSA/CKA)** → Rocky Linux or CentOS Stream (free RHEL clones).
- **Desktops** → Ubuntu, Mint, Fedora.

Release Cycles: LTS vs Rolling Release

- **LTS (Long-Term Support)**: fixed release, patched for years (e.g., Ubuntu 22.04 LTS supported 5 years). Stability-first — what production servers use.
- **Rolling release**: packages updated continuously (Arch, Manjaro) — latest software, higher risk of breakage.

Open Source Licensing

License	Key Trait
GPL (GNU Public License)	Copyleft — derivative works must also be open-sourced (used by the Linux kernel)
MIT	Very permissive — reuse/modify/sell with attribution only
Apache 2.0	Permissive + explicit patent grant (common in cloud-native projects: Kubernetes, Terraform)
BSD	Permissive, minimal restrictions

Linux vs Unix vs macOS

Unix was the original 1970s AT&T OS. Linux is a Unix-*like* clone (not derived from Unix source code) built from scratch to follow POSIX standards. **macOS** is built on **Darwin**, which is derived from **BSD** (a true Unix descendant). So: Linux and macOS are cousins — both "Unix-like," but Linux is a clean-room reimplementation while macOS/BSD trace lineage to real AT&T Unix code.

Linux vs Windows — Fundamental Differences

Aspect	Linux	Windows
Source	Open source	Closed source
Filesystem	Single tree (`/`)	Drive letters (`C:\`, `D:\`)
Case sensitivity	Case-sensitive paths	Case-insensitive
Shell	bash/zsh, scripting-first	PowerShell/cmd
Permissions	rwx + owner/group/other	ACL-based
Package mgmt	apt/dnf/pacman	MSI/EXE installers, winget
Server dominance	Dominant on cloud/web servers	Dominant on desktop

Linux Use Cases

Servers (web, DB, app), **embedded systems** (routers, IoT, Android is Linux-kernel based), **containers** (Docker/Kubernetes nodes are virtually always Linux), **cloud** (AWS/GCP/Azure VM images are majority Linux), desktop (smaller share), **HPC/supercomputing** (nearly 100% of the Top500 supercomputers run Linux).

Linux Standard Base (LSB) & Filesystem Hierarchy Standard (FHS)

- **LSB**: a (now largely deprecated) specification aiming to standardize application binary interfaces across distros so software would "just run" anywhere.
- **FHS**: the actively-used standard defining the purpose of every top-level directory (`/etc`, `/var`, `/usr`, etc.) — covered in full in Module 3.

Module 2: Linux Installation & Boot Process

Installation Environments

- **Bare metal**: installed directly on physical hardware — maximum performance, used for production servers.
- **Virtual Machine (hypervisor-based)**: VirtualBox, VMware, **KVM** (native Linux hypervisor), Hyper-V (Windows) — used for labs/testing.
- **Cloud instances**: AWS EC2, GCP Compute Engine, Azure VMs — pre-built AMIs/images, boot in seconds, billed per use.
- **Local lightweight labs**: **Multipass** (Canonical's quick Ubuntu VM tool) and **Vagrant** (declarative VM provisioning via `Vagrantfile`) — ideal for spinning up disposable practice environments.

Disk Partitioning

MBR vs GPT

	MBR (Master Boot Record)	GPT (GUID Partition Table)
Max partitions	4 primary (or 3 + extended)	128 by default
Max disk size	2 TB	9.4 ZB (practically unlimited)
Boot firmware	BIOS	UEFI (also works with BIOS via hybrid)
Redundancy	None	Backup partition table at end of disk

Standard partition layout: `/` (root), `/boot` (kernel + GRUB files, often separate so it mounts before encryption/LVM is unlocked), `/home` (user data), `/var` (logs/variable data — separated so logs can't fill root), `/tmp` (temp files), `swap` (virtual memory overflow).

LVM-based installation: instead of fixed partitions, disks become Physical Volumes → pooled into Volume Groups → sliced into resizable Logical Volumes. This is the default on RHEL-family installers because it allows **online resizing** later (see Module 15).

BIOS vs UEFI

- **BIOS**: legacy firmware, 16-bit, boots via MBR, slower POST.
- **UEFI**: modern firmware, supports GPT, Secure Boot, faster boot, graphical pre-boot environment.

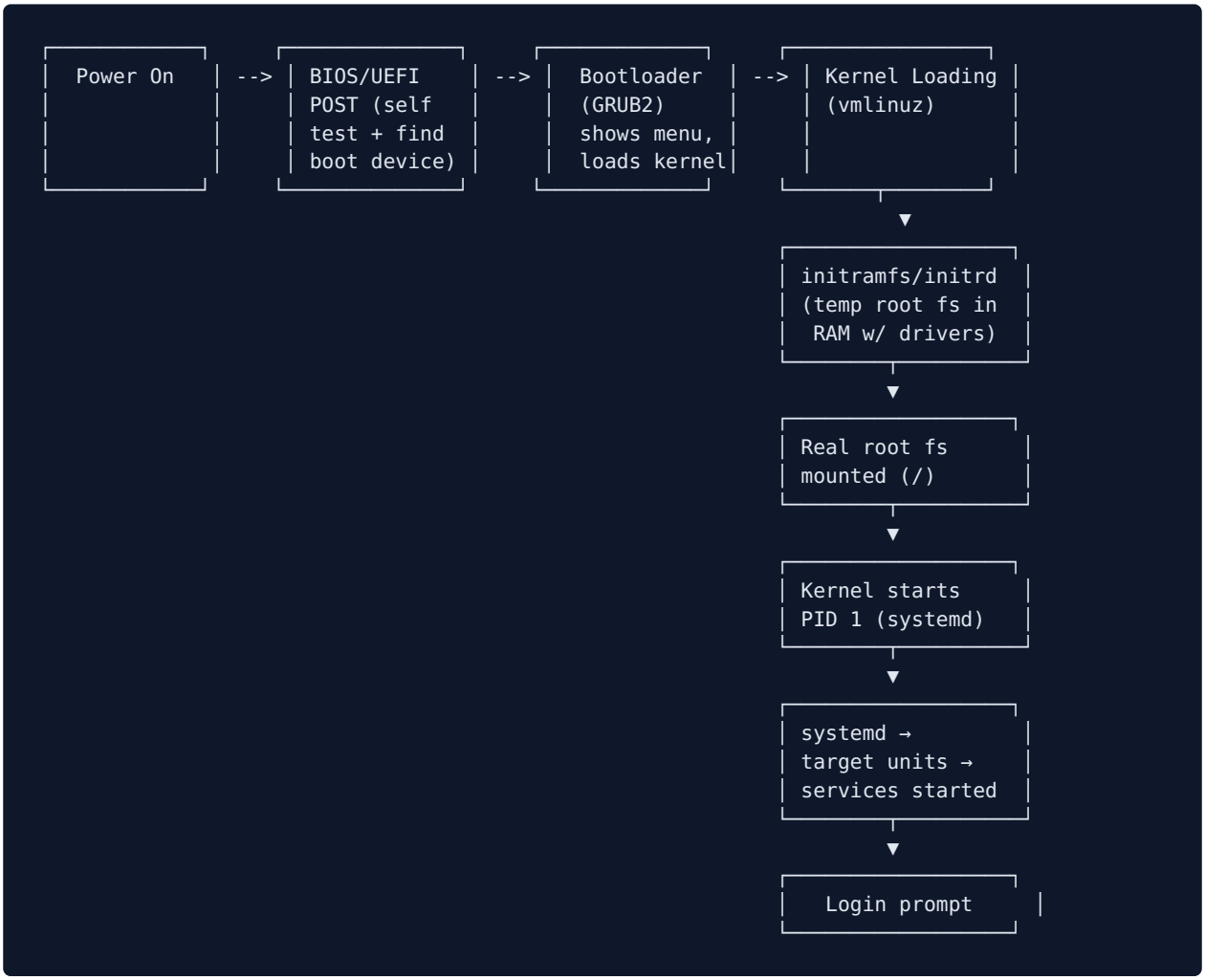
Bootloader — GRUB2

GRUB2 (**GR**and **U**nified **B**ootloader) is the standard Linux bootloader responsible for presenting a boot menu and handing control to the kernel.

```
Key files/commands:
/boot/grub2/grub.cfg        → generated config (DO NOT hand-edit directly)
/etc/default/grub           → human-edited settings (GRUB_TIMEOUT, GRUB_CMDLINE_LINUX)
grub2-mkconfig -o /boot/grub2/grub.cfg   → regenerates grub.cfg from /etc/default/grub + /
etc/grub.d/
```

- **Kernel selection at boot**: GRUB menu lets you pick among installed kernels (useful after a kernel update, in case the new one fails to boot).
- **GRUB2 rescue mode**: a minimal `grub rescue>` shell appears when GRUB cannot find its config/files — typically fixed by booting a live ISO and running `grub2-install` + `grub2-mkconfig`.

Linux Boot Process — Detailed Flow



initramfs is a small, temporary root filesystem loaded entirely into RAM. Its job: load the kernel modules needed to mount the *real* root filesystem (e.g., disk/RAID/LVM/encryption drivers) before the actual OS takes over. Once the real root is mounted, control is handed off (`switch_root`) and initramfs is discarded.

Kernel Parameters at Boot

Passed via the GRUB menu line (edit with `e` at boot, or permanently in `/etc/default/grub`'s `GRUB_CMDLINE_LINUX`). Common ones: `quiet` (suppress boot messages), `single` / `1` (single-user mode), `rd.break` (drop into initramfs shell for rescue), `selinux=0` (disable SELinux for troubleshooting).

Boot Troubleshooting Modes

- **Rescue mode**: mounts root filesystem, minimal services — used to fix boot-blocking issues.
- **Emergency mode**: even more minimal than rescue; root filesystem mounted read-only — used when rescue mode itself fails (e.g., broken `/etc/fstab`).

- **Single-user mode**: boots to a root shell without full multi-user services — classic quick-fix mode (e.g., resetting a lost root password).

Module 3: Linux Filesystem Hierarchy

Linux organizes everything under a **single root tree** (`/`) — there are no separate drive letters. Every disk, partition, or remote share is *mounted* onto a directory within this tree.

The Filesystem Hierarchy Standard (FHS) Map

```
/
├── bin, sbin           → essential binaries (merged into /usr/bin on modern distros)
├── boot                 → kernel, initramfs, GRUB files
├── dev                  → device files (disks, terminals, /dev/null)
├── etc                  → system-wide configuration files
├── home                  → regular users' personal directories
├── root                  → root user's home directory
├── lib, lib64            → essential shared libraries
├── media, mnt            → removable media / temporary mount points
├── opt                   → optional third-party software (self-contained bundles)
├── proc                  → virtual fs exposing kernel & process info
├── run                   → runtime data since boot (replaces old /var/run)
├── srv                   → data served by this system (web/ftp content)
├── sys                   → virtual fs exposing kernel/hardware/driver info
├── tmp                   → temporary files, cleared on reboot
├── usr                    → user-land programs, libraries, docs (the "real" OS)
│   ├── bin, sbin          → non-essential user/system binaries
│   ├── lib                → libraries for /usr/bin, /usr/sbin binaries
│   ├── local               → locally compiled/installed software
│   └── share               → architecture-independent data (man pages, icons)
└── var                    → variable/changing data
    ├── log                → system logs
    ├── spool              → print & mail queues
    ├── cache              → application cache
    └── lib                → persistent application state (e.g., databases)
```

Directory-by-Directory Reference

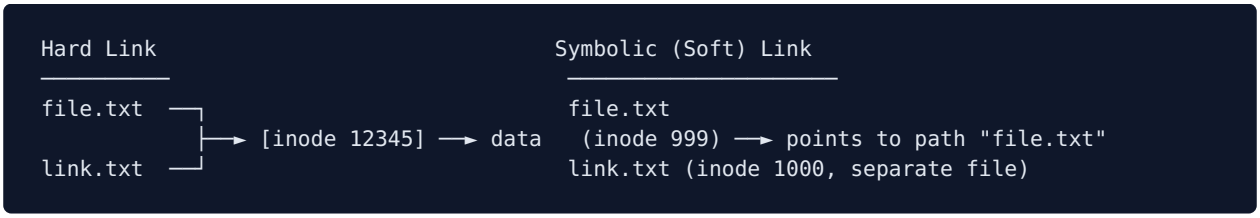
Path	Purpose
`/`	Root of the entire filesystem tree
`/bin`	Essential user binaries (`ls`, `cp`, `bash`) — needed even in single-user mode
`/sbin`	Essential system admin binaries (`fsck`, `reboot`, `ip`)
`/usr`	Bulk of installed software; “Unix System Resources,” not “user”
`/usr/bin`, `/usr/sbin`	Non-essential binaries installed via package manager
`/usr/lib`	Libraries backing `/usr/bin` & `/usr/sbin` programs
`/usr/local`	Software you compiled/installed manually (won’t be touched by package manager updates)
`/usr/share`	Architecture-independent shared data: man pages, docs, icons, timezone data
`/etc`	**All** system configuration — `/etc/passwd`, `/etc/fstab`, `/etc/ssh/sshd_config`, etc.
`/home`	One subdirectory per regular user (`/home/shaam`)
`/root`	Root user’s home (kept outside `/home` so it’s accessible even if `/home` is a separate unmounted partition)
`/var`	Data that changes at runtime: logs, spools, caches
`/var/log`	**Most important directory for troubleshooting** — all system/service logs
`/var/spool`	Queued jobs (print jobs, outgoing mail, cron)
`/var/cache`	Downloaded package caches, rendering caches
`/var/lib`	Persistent state for applications/services (e.g., `/var/lib/mysql`, `/var/lib/docker`)
`/tmp`	Temp files — **wiped on reboot** (or periodically by `systemd-tmpfiles`)
`/run`	

Path	Purpose
	Runtime data since last boot (PID files, sockets) — tmpfs, cleared on reboot
`/boot`	`vmlinuz` (kernel), `initramfs`, GRUB config — must exist before anything else mounts
`/dev`	Device nodes — every piece of hardware appears as a file here (`/dev/sda`, `/dev/null`, `/dev/tty`)
`/proc`	Virtual, in-memory filesystem exposing live kernel & process data (`/proc/cpuinfo`, `/proc/[pid]/status`)
`/sys`	Virtual filesystem exposing kernel objects/drivers/devices in a structured tree (used heavily for cgroups, see Module 22)
`/mnt`	Conventional, generic temporary mount point for admins
`/media`	Auto-mount point for removable media (USB drives, CD-ROMs)
`/opt`	Self-contained third-party application bundles (e.g., `/opt/google/chrome`)
`/lib`, `/lib64`	Essential shared libraries needed by `/bin` and `/sbin` binaries
`/srv`	Data for services this system provides (web content, FTP files)

Absolute vs Relative Paths

- **Absolute path**: starts from `/` — always unambiguous. Example: `/home/shaam/project/app.py`
- **Relative path**: starts from the current working directory. Example: if you're in `/home/shaam`, then `project/app.py` refers to the same file.

Hard Links vs Symbolic Links



	Hard Link	Symbolic Link
Points to	Same inode (same data on disk)	Path/filename of the target
Cross filesystem?	No	Yes
Link to directory?	No (generally disallowed)	Yes
If original deleted	Data still accessible via hard link	Link becomes “broken”/ dangling
Command	`ln source link`	`ln -s source link`

Special File Types

Linux represents everything as a “file.” The 7 types, visible as the first character of `ls -l` output: | Symbol | Type | |—|—| | `-` | Regular file | | `d` | Directory | | `l` | Symbolic link | | `b` | Block device (e.g., disks — buffered, random access) | | `c` | Character device (e.g., keyboards, serial ports — unbuffered, stream) | | `s` | Socket (inter-process communication endpoint) | | `p` | Named pipe / FIFO (inter-process communication channel) |

PART 2: COMMAND LINE MASTERY

Module 4: Shell & Terminal Basics

What is a Shell

A shell is a command interpreter that sits between you and the kernel — it reads what you type, parses it, and executes programs. It is both an interactive REPL and a full scripting language.

Shell Types

Shell	Notes
`bash`	**Bourne Again Shell** — default on most Linux distros
`sh`	POSIX Bourne shell (often symlinked to `dash` or `bash` in POSIX mode)
`zsh`	Highly configurable, default on macOS since Catalina
`fish`	Friendly interactive shell, modern defaults, not POSIX-compliant
`ksh`	Korn shell — common in older Unix/enterprise scripting
`tcsh`	C-shell derivative
`dash`	Minimal POSIX shell, used as `/bin/sh` on Debian/Ubuntu for fast script execution

Shell Prompt Anatomy

A typical bash prompt: `user@hostname:~/project$` - `user` — current logged-in user - `hostname` — machine name - `~/project` — current working directory (`~` = home) - `$` — regular user privilege indicator; `#` means you are root

Command Syntax

```
command [options] [arguments]
# Example:
ls -l /var/log
#  ^    ^    ^
#  cmd  opt  argument
```

- **Short options**: single dash, single letter, combinable — `ls -la`
- **Long options**: double dash, descriptive — `ls --all --human-readable`

Getting Help

```
man ls              # full manual page
man 5 passwd        # section 5 (file formats) specifically
info coreutils      # GNU's hyperlinked manual system
ls --help           # quick built-in usage summary
whatis ls           # one-line description
apropos partition   # search man page descriptions by keyword
type cd             # shows if 'cd' is a builtin, alias, or external binary
which python3        # path to the external binary that would run
whereis python3       # binary + man page + source locations
```

Man page sections: 1 = user commands, 2 = system calls, 3 = library calls, 4 = special files (`/dev`), 5 = file formats/conventions, 6 = games, 7 = miscellaneous, 8 = admin/root commands.

Command History

```
history              # show command history
Ctrl+R                # reverse incremental search through history
!!                    # re-run last command
!42                   # re-run history entry #42
!ssh                  # re-run most recent command starting with "ssh"
echo $HISTSIZE $HISTFILESIZE     # in-memory vs on-disk history size
echo $HISTFILE                  # usually ~/.bash_history
```

Tab Completion & Keyboard Shortcuts

Tab completion auto-completes commands, paths, and (with bash-completion package) flags/arguments.

Shortcut	Action
`Ctrl+C`	Kill the running foreground process
`Ctrl+Z`	Suspend foreground process (to background, stopped)
`Ctrl+D`	End input / logout of shell
`Ctrl+L`	Clear screen
`Ctrl+A` / `Ctrl+E`	Jump to start / end of line
`Ctrl+W`	Delete word before cursor
`Ctrl+U` / `Ctrl+K`	Delete from cursor to start / end of line

Terminal Multiplexers — tmux

tmux lets you keep sessions alive across SSH disconnects, and split a terminal into multiple panes.

```
tmux new -s build          # start new named session
tmux ls                    # list sessions
tmux attach -t build        # reattach to a session
# Prefix key (default Ctrl+b), then:
#   c = new window, % = split vertical, " = split horizontal
#   d = detach session, arrow keys = move between panes
```

Config lives at `~/.tmux.conf`. `screen` is an older alternative with similar session/detach concepts.

Module 5: File & Directory Operations

```
pwd                          # print working directory
ls -lah                      # long, all (incl. hidden/dot files), human-readable sizes
ls -lhR                      # recursive long listing
ls -lt                       # sort by modification time
ls -lS                       # sort by size
ls -li                       # show inode numbers
cd /var/log                  # change directory
cd ~    cd -    cd ..   cd /     # home, previous dir, parent, root
mkdir -p a/b/c               # create nested dirs in one shot
rmdir empty_dir              # remove only if empty
touch newfile.txt            # create empty file / update mtime
cp -r src/ dst/              # copy recursively
cp -a src/ dst/              # archive copy: preserves perms, timestamps, links
mv old.txt new.txt           # rename or move
rm -rf folder/               # remove recursively, force (NO confirmation — be
careful!)
rm -i file.txt               # interactive confirm before delete
ln source.txt hardlink.txt   # hard link
ln -s /path/to/source symlink        # symbolic link
```

Long Listing Format Interpretation

```
-rwxr-xr--  1 shaam devops  4096 Oct  4 09:12 deploy.sh
|└┘└┘└┘  |   |     |       |        |          |
| |  |  |   |     |       |        |          └─ filename
| |  |  |   |     |       |        └─ modification time
| |  |  |   |     |       └─ size in bytes
| |  |  |   |     └─ group owner
| |  |  |   └─ user owner
| |  |  └─ link count
| |  └─ other's permissions (r--)
| └─ group's permissions (r-x)
└─ owner's permissions (rwx)
file type (- regular, d directory, l symlink...)
```

find — the Swiss-army search command

```
find /var/log -name "*.log"             # by name (case-sensitive)
find . -iname "readme*"                 # case-insensitive
find / -type f                          # files only;  -type d for directories, -type l
for symlinks
find . -size +100M                      # larger than 100MB
find /var/log -mtime -7                 # modified in last 7 days
find / -perm -4000                      # SUID files
find / -user shaam -group devops        # owned by user AND group
find . -name "*.tmp" -delete            # find + delete in one go
find . -name "*.sh" -exec chmod +x {} \; # run a command per result
find /tmp -type f -mtime +30 -not -name "*.keep"   # logical NOT
```

locate vs find

```
updatedb              # rebuild the locate database (usually via cron)
locate nginx.conf      # instant search — but relies on a (possibly stale) index
```

`locate` is fast but can miss very recent files until the database refreshes; `find` is slower but always live-accurate.

File Metadata & Disk Usage

```
stat file.txt           # inode, size, timestamps, permissions — full metadata
file ./mystery_binary    # detect actual file type (not just by extension)
tree /etc -L 2           # visual directory tree, 2 levels deep
du -sh /var/log          # total size of a directory, human-readable
du -ah --max-depth=1 /home   # size of each item one level deep
df -hT                   # free space per mounted filesystem + its type
df -i                    # inode usage (can run out of inodes even with free space!)
```

Module 6: File Content Operations

```
cat -n file.txt              # number all lines
cat -A file.txt               # show hidden chars (tabs as ^I, line ends as $)
cat file1 file2 > combined.txt  # concatenate files

less file.txt                  # page through (searchable, doesn't load whole file)
# inside less: space=next page, b=back, g=top, G=bottom, /pattern=search fwd,
#              ?pattern=search back, n/N=next/prev match, q=quit
more file.txt                   # older pager, forward-only

head -n 20 file.txt             # first 20 lines
tail -n 20 file.txt              # last 20 lines
tail -f /var/log/syslog           # follow in real time — classic log-watching command
tail -F /var/log/app.log           # follow by name — survives log rotation

wc -l file.txt       # line count
wc -w file.txt        # word count
wc -c file.txt         # byte count

sort file.txt                  # alphabetic sort
sort -n numbers.txt              # numeric sort
sort -k2 -t, data.csv             # sort by 2nd comma-delimited field
sort -u file.txt                   # sort + remove duplicates

uniq -c sorted.txt       # count occurrences (input MUST be sorted first)
uniq -d sorted.txt         # show only duplicated lines

cut -d',' -f1,3 data.csv    # extract fields 1 and 3, comma-delimited
cut -c1-10 file.txt           # extract characters 1-10

paste file1 file2             # merge lines side by side
join file1 file2                # join on common field (both sorted first)

diff -u old.conf new.conf        # unified diff format (used for patches)
diff -y old.conf new.conf          # side-by-side diff
patch < changes.diff                 # apply a diff

tr 'a-z' 'A-Z' < file.txt      # lowercase → uppercase
tr -d '\r' < winfile.txt         # delete carriage returns (CRLF → LF)
tr -s ' '                         # squeeze repeated spaces into one

od -c file.bin           # octal dump
xxd file.bin               # hex dump (more human-friendly than od)
strings suspicious.bin        # extract printable strings from a binary
iconv -f ISO-8859-1 -t UTF-8 file.txt > out.txt   # encoding conversion
```

Module 7: Text Processing — grep, sed, awk

grep — Pattern Searching

```
grep -i "error" app.log          # case-insensitive
grep -v "DEBUG" app.log            # invert match (show non-matching lines)
grep -r "TODO" ./src                # recursive through a directory
grep -l "ERROR" *.log                # just list filenames with a match
grep -n "fail" app.log                 # show line numbers
grep -c "WARN" app.log                  # count matching lines
grep -o -E "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" access.log   # extract only the match (IP
addresses)
grep -A 3 -B 2 "Exception" app.log       # 3 lines After, 2 lines Before context
grep -E "error|warn|fail" app.log          # Extended regex, alternation
grep -P '(?<=user=)\w+' app.log              # Perl-compatible regex (lookbehind)
grep -w "cat" file.txt                         # whole-word match only
```

`grep` = Basic Regex by default; `egrep` / `grep -E` = Extended Regex (adds `+ ? | ()` without escaping); `fgrep` / `grep -F` = fixed string, no regex at all (fastest for literal searches).

Regular Expressions Cheat Sheet

Symbol	Meaning
`.`	Any single character
`*`	Zero or more of preceding
`+`	One or more (ERE/PCRE)
`?`	Zero or one (ERE/PCRE)
`^` / `$`	Start / end of line
`[abc]` / `[a-z]` / `[^abc]`	Character class / range / negated class
`[[:digit:]]`, `[[:alpha:]]`, `[[:space:]]`	POSIX classes
`{n}`, `{n,}`, `{n,m}`	Exact / minimum / range repetition
`()`	Grouping
`|`	Alternation (OR)
`\1`	Backreference to group 1
`(?=...)`, `(?<=...)`	Lookahead / lookbehind (PCRE only)

Greedy (`*`, default) matches as much as possible; lazy (`*?` in PCRE) matches as little as possible.

sed — Stream Editor

```
sed 's/foo/bar/' file.txt          # replace first occurrence per line
sed 's/foo/bar/g' file.txt           # replace ALL occurrences per line (global)
sed 's/foo/bar/gi' file.txt            # global + case-insensitive
sed -n '5,10p' file.txt                  # print only lines 5-10
sed '/^#/d' file.conf                      # delete comment lines
sed -n '/ERROR/p' app.log                    # print only matching lines
sed '3a\New line after line 3' file.txt        # append after line 3
sed '1i\New first line' file.txt                 # insert before line 1
sed '2c\Replacement line' file.txt                 # change (replace) line 2
sed -e 's/a/1/' -e 's/b/2/' file.txt                 # multiple commands
sed -i 's/old/new/g' file.txt                          # in-place edit (modifies file
directly!)
sed -i.bak 's/old/new/g' file.txt                       # in-place with backup
sed -f script.sed file.txt                                # commands from a script file
```

awk — Pattern Scanning & Reporting

```
awk '{print $1, $3}' file.txt                # print fields 1 and 3 (whitespace-delimited)
awk -F',' '{print $2}' data.csv                # custom field separator
awk '{print NR, NF, $0}' file.txt                # NR=record#, NF=field count, $0=whole
line
awk '/error/ {count++} END {print count}' app.log   # pattern + action + END block
awk 'BEGIN{print "Report:"} {sum+=$2} END{print sum}' data.txt
awk '{ if ($3 > 90) print $1, "CRITICAL" }' usage.txt     # conditionals
awk '{ for(i=1;i<=NF;i++) print $i }' file.txt              # loop over fields
awk '{arr[$1]++} END{for (k in arr) print k, arr[k]}' access.log   # arrays — count by key
awk '{print toupper($1), length($2)}' file.txt                     # built-in functions
awk '{printf "%-10s %5d\n", $1, $2}' file.txt                        # printf formatting
```

Rule of thumb: **sed** for line-based find/replace and editing; **awk** for column/field-based processing, math, and reports.

xargs — Building Commands from Input

```
find . -name "*.log" | xargs rm                    # delete all matched files
find . -name "*.txt" | xargs -I{} mv {} {}.bak       # placeholder substitution
find . -name "*.jpg" | xargs -n 1 -P 4 convert ...     # 4 parallel processes, 1 arg each
find . -name "*.txt" -print0 | xargs -0 cat             # null-delimited — safe for
filenames with spaces
```

Module 8: I/O Redirection & Pipes

Standard Streams

FD	Stream	Default destination
0	stdin	Keyboard
1	stdout	Terminal
2	stderr	Terminal

Redirection Operators

```
cmd > out.txt          # redirect stdout, overwrite
cmd >> out.txt           # redirect stdout, append
cmd 2> err.txt             # redirect stderr only
cmd 2>> err.txt              # redirect stderr, append
cmd &> all.txt                 # redirect BOTH stdout and stderr
cmd > out.txt 2>&1               # stdout to file, then stderr follows stdout (order
matters!)
cmd 1>/dev/null                   # discard stdout
cmd 2>/dev/null                     # discard stderr (common: "silence warnings")
cmd < input.txt                       # feed a file as stdin
cmd <<EOF
line1
line2
EOF                                      # here-document — multi-line stdin block
cmd <<< "single line"                     # here-string — one line as stdin
```

Pipes

```
ps aux | grep nginx                # pipe stdout of ps into grep
cmd1 |& cmd2                         # pipe BOTH stdout+stderr
cat file | sort | uniq -c | sort -rn   # pipeline chaining
mkfifo mypipe                          # named pipe (FIFO) — persists as a file
```

tee — Split Output

```
command | tee output.txt           # write to file AND still show on screen
command | tee -a output.txt          # append mode
command | tee file1 file2              # write to multiple files at once
```

Process Substitution

```
diff <(ls dir1) <(ls dir2)          # treat command output as a temporary "file" input
cmd > >(gzip > out.gz)                 # treat a command as an output "file"
```

Special Devices

Device	Purpose
`/dev/null`	Discards anything written to it — “the black hole”
`/dev/zero`	Infinite stream of null bytes (used to create fixed-size files)
`/dev/random`	Blocking random byte source (true entropy, can stall)
`/dev/urandom`	Non-blocking random byte source (preferred for most uses)
`/dev/stdin/stdout/stderr`	File-path aliases for the current process’s streams

Module 9: Shell Scripting

Script Basics

```
#!/bin/bash
# Shebang line — tells the kernel which interpreter runs this file
```

```
chmod +x script.sh      # make executable
./script.sh               # run (needs execute permission + shebang)
bash script.sh              # run explicitly with bash, ignores shebang/permission
source script.sh              # run IN the current shell (variables persist after)
```

Variables

```
name="Shaam"              # NO spaces around =
echo $name                  # access
echo ${name}                  # access (braces avoid ambiguity, e.g. ${name}_suffix)
readonly PI=3.14                # cannot be changed after this
unset name                        # remove a variable
export MY_VAR="value"               # make visible to child processes (environment
variable)
```

Special Variables

Variable	Meaning
`$0`	Script name
`$1` … `$9`	Positional arguments
`$@`	All arguments, each as a separate quoted string
`$*`	All arguments as a single string
`$#`	Number of arguments
`$?`	Exit status of the last command (0 = success)
`$$`	PID of current script
`$!`	PID of last backgrounded process
`$_`	Last argument of the previous command

Quoting

```
echo 'Price is $5'        # single quotes → literal, NO expansion
echo "User is $USER"        # double quotes → variables/commands ARE expanded
echo "Today: $(date)"         # $() preferred modern command substitution
echo "Today: `date`"            # backticks — older, harder to nest
echo "Tab:\tNewline:\n"           # \ is the escape character
```

Arithmetic

```
echo $((5 + 3))          # arithmetic expansion
let x=5+3                  # older syntax
expr 5 + 3                   # oldest syntax (needs spaces)
echo "scale=2; 10/3" | bc      # floating point via bc
```

String Operations

```
str="Hello World"
echo ${#str}                    # length → 11
echo ${str:0:5}                   # substring → "Hello"
echo ${str/World/Linux}             # replace first match → "Hello Linux"
echo ${str#Hello }                    # strip shortest match from front
echo ${str%World}                       # strip shortest match from back
echo ${undefined:-"default"}              # use default if unset
echo ${undefined:="default"}                # use default AND assign it
```

Conditionals

```
if [ -f "$file" ]; then
  echo "exists"
elif [ -d "$file" ]; then
  echo "is a directory"
else
  echo "not found"
fi

[[ "$name" == "shaam" && -n "$name" ]]   # extended test — supports &&, ||, regex =~

case "$1" in
  start) echo "starting" ;;
  stop)  echo "stopping" ;;
  *)     echo "usage: $0 {start|stop}" ;;
esac
```

File test flags: `-f` regular file, `-d` directory, `-e` exists, `-r/-w/-x` read/write/execute, `-s` size > 0, `-L` symlink, `-z`/`-n` empty/non-empty string. **Integer comparisons**: `-eq -ne -lt -le -gt -ge`. **String comparisons**: `= != < >`.

Loops

```
for i in 1 2 3; do echo $i; done
for ((i=0; i<5; i++)); do echo $i; done
while read -r line; do echo "$line"; done < file.txt
until [ "$x" -ge 10 ]; do x=$((x+1)); done
for f in *.log; do [ "$f" == "skip.log" ] && continue; echo "$f"; done   # continue
while true; do ...; break; done                                           # break / infinite loop
```

Functions

```
greet() {
  local name=$1              # 'local' keeps the variable function-scoped
  echo "Hello, $name"
  return 0                    # exit code, NOT a return value
}
greet "Shaam"                   # call
result=$(greet "Shaam")           # capture "output" via command substitution
```

Arrays

```
arr=(apple banana cherry)
echo ${arr[1]}              # banana
echo ${#arr[@]}               # length: 3
arr+=(date)                     # append
for item in "${arr[@]}"; do echo "$item"; done   # iterate

declare -A map                   # associative array
map[name]="Shaam"
map[role]="DevOps"
echo ${map[role]}
```

Input/Output

```
read -p "Enter name: " name       # prompt
read -s -p "Password: " pass        # silent (hidden) input
read -t 5 -p "Quick! " ans             # 5-second timeout
read -a arr -p "List: "                  # read into an array
printf "%-10s %5d\n" "cpu" 42               # printf — precise formatting (unlike echo)
```

Error Handling

```
set -e            # exit immediately if any command fails
set -u             # exit if an undefined variable is used
set -o pipefail      # a pipeline fails if ANY command in it fails, not just the last
set -x                # debug mode — print every command before executing it
trap 'echo "cleanup running"; rm -f /tmp/lock' EXIT    # run on script exit (success OR
failure)
command || { echo "failed" >&2; exit 1; }                # send errors to stderr
```

Debugging

```
bash -n script.sh      # syntax check only, doesn't execute
bash -x script.sh        # trace every command as it runs
bash -v script.sh          # print script lines as read
shellcheck script.sh         # static analysis — catches quoting bugs, unsafe patterns
```

Script Best Practices

A production-grade script should include: a header comment (purpose, author, usage), a `usage()` function for `--help`, argument validation before doing anything destructive, `mktemp` for temp files, a `trap ... EXIT` cleanup handler, `flock` to prevent two copies running concurrently, structured logging (timestamped, to a log file), and should be tested for portability if it must also run under plain `sh`.

PART 3: SYSTEM ADMINISTRATION

Module 10: User & Group Management

User Types

- **root** — UID 0, unrestricted superuser.
- **System users** — UID 1-999, created automatically by services (e.g., `nginx`, `mysql`) to run daemons without full privileges.
- **Regular users** — UID 1000+, human accounts.

Key Account Files

```
/etc/passwd   → username:password(x):UID:GID:GECOS:home:shell
/etc/shadow   → username:hashed-password:last-change:min:max:warn:inactive:expire
/etc/group    → groupname:password:GID:comma-separated-members
/etc/gshadow  → group password hashes + administrators
```

The `x` in `/etc/passwd` means the real hash lives in `/etc/shadow` (readable only by root) — this is what separates legacy world-readable password storage from modern secure storage.

User Management Commands

```
useradd -m -d /home/shaam -s /bin/bash -u 1050 -g devops -G docker,sudo -c "Shaam N" shaam
#        -m create home  -d home path  -s shell  -u UID  -g primary grp  -G secondary grps
-c comment
usermod -aG docker shaam         # -aG = append to group (NEVER use -G alone — it
overwrites all groups!)
usermod -L shaam                   # lock account (-U to unlock)
usermod -e 2026-12-31 shaam           # set account expiry date
userdel -r shaam                       # delete user AND home directory

passwd shaam                # set/change password interactively
passwd -l shaam                # lock password
passwd -S shaam                  # show password status
chage -l shaam                     # list password aging info
chage -M 90 -W 7 shaam               # max 90 days validity, 7-day warning

id shaam                 # show UID, GID, and group memberships
whoami                     # current user
who                          # who is logged in
w                              # logged in users + what they're doing
last                             # login history (from /var/log/wtmp)
lastlog                            # last login time per user
finger shaam                         # user information (if installed)
```

New accounts inherit defaults from `/etc/default/useradd` and get starter dotfiles copied from `/etc/skel`.

Group Management

```
groupadd devops
groupmod -n developers devops      # rename
groupdel developers
gpasswd -a shaam devops              # add member via group tools
newgrp devops                          # switch active group for current session
groups shaam                             # list a user's groups
```

su vs sudo

```
su - shaam        # switch user WITH their full login environment (the "-" matters!)
su shaam             # switch user but KEEP current shell's environment
sudo -i                # root shell with root's environment
sudo -u postgres psql   # run a single command as another user
sudo -l                  # list what the current user is permitted to run
visudo                     # SAFELY edit /etc/sudoers (syntax-checks before saving)
```

`/etc/sudoers` syntax: `user host=(runas_user) command_list`. Example: `shaam ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx`. Prefer dropping custom rules into `/etc/sudoers.d/` rather than editing the main file.

PAM (Pluggable Authentication Modules)

PAM decouples “how do I authenticate” from the applications themselves. Config lives in `/etc/pam.d/` (one file per service, e.g., `sshd`, `login`).

```
auth      required   pam_unix.so     # module types: auth, account, session, password
```

Control flags: `required` (must pass, but continues evaluating other rules), `requisite` (must pass, stops immediately on failure), `sufficient` (passing is enough to grant access), `optional` (doesn’t affect outcome unless it’s the only module). Common modules: `pam_unix` (traditional password check), `pam_limits` (enforces `/etc/security/limits.conf`), `pam_wheel` (restrict `su` to the `wheel` group), `pam_faillock` / `pam_tally2` (lock account after repeated failed logins).

Resource Limits & Login Restrictions

```
ulimit -n            # show max open file descriptors for current shell
ulimit -a              # show all current limits
```

`/etc/security/limits.conf` sets persistent per-user/group limits (e.g., `shaam soft nofile 4096`). `/etc/securetty` restricts which terminals root may log in from; `/etc/nologin`, if present, blocks all non-root logins (used during maintenance).

Module 11: File Permissions & Access Control

Standard Permissions

```
          rwx r-x r--
          |   |   |
          |   |   └── others (everyone else)
          |   └────── group
          └────────── owner
```

read=4, write=2, execute=1 → sum per category gives octal notation (e.g., `rwxr-xr--` = **754**).

For a **directory**: `r` = can list contents, `w` = can create/delete entries inside, `x` = can `cd` into it / traverse it.

```
chmod u+x script.sh            # add execute for owner (symbolic mode)
chmod g-w file.txt               # remove write for group
chmod o=r file.txt                 # set exactly read for others
chmod a+x script.sh                  # add execute for everyone
chmod 755 script.sh                    # octal mode: rwxr-xr-x
chmod -R 644 /var/www/html               # recursive

chown shaam:devops file.txt       # change owner AND group
chown -R shaam:devops /app          # recursive
chgrp devops file.txt                 # change group only
```

umask — Default Permission Mask

```
umask           # show current mask, e.g., 0022
umask 027        # set a new mask (typically in ~/.bashrc or /etc/profile)
```

Default max permissions are 666 for files and 777 for directories; `umask` is **subtracted**. A umask of `022` turns a would-be 777 directory into `755`, and a would-be 666 file into `644`. Directories and files differ because files never get execute permission by default (only directories/programs do).

Special Permission Bits

```
chmod 4755 /usr/bin/passwd
# SUID (4xxx) — 'passwd' runs as root even when a normal user runs it
chmod 2775 /shared/project      # SGID (2xxx) — new files inside inherit the directory's
group
chmod 1777 /tmp
# Sticky bit (1xxx) — only the file's owner (or root) can delete it
find / -perm -4000 2>/dev/null       # find all SUID binaries (classic security audit step)
```

- **SUID** on an executable: runs with the **file owner's** privileges, not the caller's — a major privilege-escalation risk if misused.
- **SGID** on an executable: runs with the **file's group** privileges; on a **directory**, all new files/subdirs automatically inherit that directory's group (used heavily for team shared folders).
- **Sticky bit**: classic use is `/tmp` — everyone can create files, but only the owner can delete their own.

Access Control Lists (ACLs)

Standard permissions only support one owner + one group. ACLs let you grant fine-grained access to **additional** specific users/groups.

```
getfacl file.txt                      # view ACLs
setfacl -m u:shaam:rwx file.txt         # grant user 'shaam' rwx in addition to normal
perms
setfacl -m g:devops:rx file.txt           # grant a group read+execute
setfacl -x u:shaam file.txt                 # remove a specific entry
setfacl -b file.txt                           # strip ALL ACL entries
setfacl -R -m u:shaam:rwx /project               # recursive
setfacl -d -m u:shaam:rwx /project                 # default ACL — inherited by new files
created inside
```

The **mask** entry caps the maximum effective permissions any named user/group ACL entry can actually exercise — even if an entry says `rwx`, if the mask is `r-x`, write is blocked. Filesystems must be mounted with `acl` support (ext4/XFS support it by default on modern kernels).

Extended Attributes & Immutability

```
chattr +i important.conf     # make a file immutable — even root can't delete/modify it
until unset
chattr +a logfile.log           # append-only — can grow, can't be truncated/overwritten
lsattr file.txt                    # list special attributes
getfattr -d file.txt                 # view generic extended attributes
setfattr -n user.comment -v "note" file.txt
```

File Capabilities

Instead of giving a binary full root via SUID, Linux **capabilities** grant a narrow slice of root's power.

```
getcap /usr/bin/ping                    # view capabilities on a binary
setcap cap_net_bind_service=+ep /usr/bin/myserver   # allow binding to ports <1024 without
being root
```

Common capabilities: `CAP_NET_BIND_SERVICE` (bind privileged ports), `CAP_NET_RAW` (raw sockets, used by `ping`).

Module 12: Process Management

Viewing Processes

```
ps aux                  # BSD-style: ALL processes, every user
ps -ef                    # UNIX-style: full format listing
ps -o pid,ppid,cmd,%cpu,%mem -p 1234    # custom columns for one PID
ps -u shaam                # processes owned by a specific user
```

Key `ps aux` columns: USER, PID, %CPU, %MEM, VSZ/RSS (memory), STAT (state), START, TIME, COMMAND.

```
top                   # interactive, live-updating process viewer
# inside top: k=kill, r=renice, M=sort by memory, P=sort by CPU, 1=per-core view, q=quit
htop                     # enhanced, colorized, mouse-friendly top (F5 = tree view)
pgrep -l nginx              # list PIDs + names matching "nginx"
pgrep -u shaam -a              # processes for a user, with full command line
pstree -p                        # process tree with PIDs
pidof nginx                        # just the PID(s) of a running program
```

Process States

Code	State
R	Running or runnable
S	Sleeping (waiting for an event)
D	Uninterruptible sleep (usually I/O — can’t even be killed with SIGKILL)
Z	Zombie (finished, but parent hasn’t reaped its exit status)
T	Stopped (e.g., via `Ctrl+Z` or `SIGSTOP`)
X	Dead

Signals & Killing Processes

```
kill -l                 # list all signal names/numbers
kill 1234                 # send SIGTERM (15) — polite "please stop" request
kill -9 1234                # send SIGKILL (9) — immediate, un-catchable termination
kill -1 1234                  # SIGHUP (1) — often used to make a daemon reload its config
pkill -u shaam nginx             # kill by name/pattern + filters
pkill -f "python app.py"            # match full command line
killall nginx                         # kill all processes by exact name
```

Signal	Number	Meaning
SIGHUP	1	Hangup — often triggers config reload
SIGINT	2	Interrupt (what `Ctrl+C` sends)
SIGKILL	9	Force kill — cannot be caught, blocked, or ignored
SIGTERM	15	Graceful termination request (default for `kill`)
SIGSTOP	19	Pause process (cannot be caught)
SIGCONT	18	Resume a stopped process
SIGUSR1/2	10/12	User-defined, app-specific purposes

Always try SIGTERM (default `kill`) before SIGKILL — it allows the process to clean up (close files, flush buffers); SIGKILL gives it no chance to do so.

Job Control

```
long_task.sh &         # run in background
jobs                     # list background/suspended jobs for this shell
fg %1                      # bring job 1 to foreground
bg %1                        # resume a suspended job in the background
disown %1                      # detach a job so it survives shell exit
Ctrl+Z                           # suspend the current foreground job
nohup long_task.sh &               # immune to SIGHUP — keeps running after you log out
```

Priority Management

```
nice -n 10 long_task.sh     # start at a lower priority (higher niceness = nicer to others)
renice -n 5 -p 1234            # change priority of an already-running process
ionice -c 2 -n 7 -p 1234         # I/O scheduling priority (class 2=best-effort, level
7=lowest)
```

Nice values range **-20 (highest priority) to +19 (lowest priority)**; only root can set negative (higher-priority) values.

Inspecting Processes via /proc

```
cat /proc/1234/status       # detailed state, memory, signals
cat /proc/1234/cmdline        # exact command line used to launch it
cat /proc/1234/environ          # environment variables (null-separated)
ls -l /proc/1234/fd               # open file descriptors
cat /proc/1234/maps                 # memory mappings
```

Finding What's Using a File/Port

```
lsof -u shaam                 # open files for a user
lsof -p 1234                    # open files for a specific PID
lsof -i :8080                     # what's listening on port 8080
lsof /var/log/app.log               # which processes have this file open
fuser -v /mnt/data                    # verbose — who's using a mount point
fuser -k /mnt/data                      # kill everything using it (careful!)
fuser -n tcp 8080                         # find process bound to a TCP port
```

System Call & Library Tracing

```
strace -p 1234                  # attach and trace syscalls of a running process
strace -f -e trace=open,read ls   # trace only specific syscalls, follow forks
strace -c ls                        # summary/counts of syscalls instead of a live trace
ltrace ./myprogram                    # trace library (not kernel) calls — e.g., libc
functions
```

`strace` is one of the most powerful troubleshooting tools: it reveals exactly which files a mysteriously-failing program is trying (and failing) to open, what permissions errors occur, etc.

Module 13: Service Management with systemd

systemd vs SysV init

systemd replaced the old sequential SysV init scripts (`/etc/init.d/`) with a parallelized, dependency-graph-based init system that is now standard on virtually all major distros (RHEL 7+, Ubuntu 15.04+, Debian 8+).

systemctl — The Control Command

```
systemctl start nginx           systemctl stop nginx
systemctl restart nginx           systemctl reload nginx     # reload config w/o dropping
connections
systemctl status nginx              # health + recent logs
systemctl enable nginx                # start automatically at boot
systemctl disable nginx                 # don't start at boot
systemctl enable --now nginx              # enable AND start immediately
systemctl mask nginx                        # hard-block — even manual start fails until
unmasked
systemctl unmask nginx
systemctl list-units --type=service             # all currently loaded service units
systemctl list-unit-files                          # all installed units + their enabled
state
systemctl list-dependencies nginx                     # what nginx depends on
systemctl daemon-reload
# re-read unit files after editing them
systemctl isolate multi-user.target                         # switch to a different target
(~runlevel)
systemctl get-default                                          # show default boot target
systemctl set-default multi-user.target                          # set default boot target
```

systemd Unit Types

Suffix	Purpose
`.service`	A long-running daemon or one-shot program
`.socket`	Socket-activated services (start on first connection)
`.timer`	Scheduled execution — modern replacement for cron
`.target`	Groups units together, like old runlevels
`.mount` / `.automount`	Filesystem mounts / on-demand mounts
`.path`	Trigger a unit when a file/path changes
`.slice`	Resource-control grouping (cgroups)
`.scope`	Externally-created process groups
`.device`	Kernel-exposed devices
`.swap`	Swap space

Unit File Locations (priority order)

```
/etc/systemd/system/     → admin-created/customized (HIGHEST priority)
/run/systemd/system/     → runtime-generated
/lib/systemd/system/     → package-installed defaults (lowest priority)
```

Anatomy of a Service Unit File

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
Documentation=https://internal-wiki/myapp
After=network.target postgresql.service
Requires=postgresql.service
Wants=redis.service

[Service]
Type=simple
ExecStart=/usr/bin/myapp --config /etc/myapp/config.yml
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
User=myapp
Group=myapp
WorkingDirectory=/opt/myapp
Environment=NODE_ENV=production
EnvironmentFile=/etc/myapp/env

[Install]
WantedBy=multi-user.target
```

Service Types: `simple` (default — process runs in foreground), `forking` (daemonizes itself), `oneshot` (runs once and exits, e.g., a setup script), `notify` (signals systemd via sd_notify when ready), `dbus`, `idle`. **Restart policy options**: `always`, `on-failure`, `on-abnormal`, `on-abort`.

```
systemctl edit nginx        # creates an override/drop-in file WITHOUT touching the vendor
unit
systemctl edit --full nginx   # edit the full unit file directly
```

systemd Targets (Runlevel Equivalents)

Target	Old Runlevel	Meaning
`poweroff.target`	0	Shutdown
`rescue.target`	1	Single-user mode
`multi-user.target`	3	Full multi-user, no GUI (typical servers)
`graphical.target`	5	Multi-user + GUI
`reboot.target`	6	Reboot
`emergency.target`	—	Absolute minimal — root shell, almost nothing mounted

systemd Timers (cron replacement)

```
# myjob.timer
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true    # if the system was off at 2 AM, run it as soon as it's back on

[Install]
WantedBy=timers.target
```

`OnCalendar` = absolute scheduling (like cron); `OnBootSec=15min` / `OnUnitActiveSec=1h` = relative scheduling. Timers are generally preferred over cron in modern systemd environments because they integrate with logging, dependencies, and `systemctl status`.

journald & journalctl

```
journalctl -u nginx                # logs for one unit only
journalctl -u nginx -f               # follow live, like tail -f
journalctl -n 100                      # last 100 lines (across all units)
journalctl -p err                        # only priority "err" and above
journalctl -b                              # since last boot
journalctl -b -1                             # since the PREVIOUS boot
journalctl --since "1 hour ago" --until now
journalctl -k                                    # kernel messages only (like dmesg)
journalctl -e                                      # jump straight to the end
journalctl -r                                        # reverse order (newest first)
journalctl --no-pager                                  # plain stdout output, good for
scripting/piping
```

`/etc/systemd/journald.conf` controls whether logs persist across reboots (`Storage=persistent`) and their max disk usage.

Boot Performance Analysis

```
systemd-analyze time                # total boot time breakdown (firmware/loader/kernel/
userspace)
systemd-analyze blame                 # slowest-starting units, ranked
systemd-analyze critical-chain          # the dependency chain that determined total boot
time
systemd-analyze plot > boot.svg           # visual boot timeline
```

Other systemd Utilities

```
systemd-cgls           # view the live cgroup hierarchy as a tree
systemd-cgtop            # top-style resource usage by cgroup
loginctl                   # manage/list login sessions
hostnamectl                  # view/set hostname
hostnamectl set-hostname web01.internal
timedatectl                    # view/set time, timezone, NTP sync status
localectl                        # view/set locale and keyboard layout
```

Module 14: Package Management

RPM-Based Systems (RHEL, CentOS, Rocky, Fedora)

```
rpm -ivh package.rpm           # install, verbose, show hash progress
rpm -Uvh package.rpm             # upgrade (or install if not present)
rpm -e package                     # erase/remove
rpm -qa                              # query ALL installed packages
rpm -qi httpd                          # query info about an installed package
rpm -ql httpd                            # list all files owned by a package
rpm -qf /etc/httpd/conf/httpd.conf          # which package owns this file?
rpm -qp --info package.rpm                    # query info on a .rpm file not yet installed
rpm -V httpd                                     # verify — detect modified/missing files
rpm --import RPM-GPG-KEY-redhat                    # import a vendor's signing key

dnf install httpd                    # high-level — resolves dependencies automatically
dnf remove httpd                       dnf update httpd       dnf upgrade
dnf search "web server"                   dnf info httpd        dnf list installed
dnf group install "Development Tools"
dnf history                                 # transaction log
dnf history undo 15                           # roll back transaction #15 — huge safety net
dnf provides */bin/kubectl                      # find which package provides a given file
dnf repolist                                      # list configured repositories
dnf config-manager --add-repo <url>                 # add a repo
dnf module list nginx                                 # RHEL 8+ application streams
(multiple versions)
dnf autoremove                                          # clean up orphaned dependencies
dnf clean all                                             # clear cached package data
```

Repo definitions live as `.repo` files under `/etc/yum.repos.d/`. GPG verification (`gpgcheck=1`) ensures packages haven't been tampered with. **rpm-ostree** is the image-based variant used by Fedora Silverblue/CoreOS — updates are applied as atomic, rollback-able OS-tree commits rather than in-place package installs.

DEB-Based Systems (Debian, Ubuntu)

```
dpkg -i package.deb         dpkg -r package        dpkg -P package   # install / remove /
purge(+configs)
dpkg -l                       # list all installed packages
dpkg -L package                 # list files owned by a package
dpkg -S /usr/bin/nginx            # which package owns this file?
dpkg --configure -a                 # finish any interrupted installs
dpkg-reconfigure tzdata                # re-run a package's config wizard

apt install nginx            apt remove nginx       apt purge nginx       apt autoremove
apt update                     # refresh package index (does NOT upgrade anything)
apt upgrade                      # upgrade packages (won't remove packages to resolve deps)
apt full-upgrade                   # upgrade + allowed to remove packages if needed
apt search nginx                      apt show nginx        apt list --installed
apt-cache policy nginx                    # see available versions + which repo they're
from
```

`/etc/apt/sources.list` and `/etc/apt/sources.list.d/*.list` define repositories. Modern key management uses `/etc/apt/trusted.gpg.d/` (the older global `apt-key` is deprecated). **Pinning** (`/etc/apt/preferences.d/`) lets you prefer specific versions/repos. `apt-mark hold nginx` prevents a package from being auto-upgraded; `apt-mark showmanual` shows what you explicitly installed vs. pulled in as a dependency.

Universal / Sandboxed Package Formats

```
snap install code --classic     # Canonical's sandboxed format, auto-updating
snap list                          snap refresh          # update all snaps
flatpak install flathub org.gimp.GIMP      # desktop-app-focused sandboxed format
```

Snap has confinement levels (`strict`, `classic`, `devmode`) controlling how much host access the app gets. Flatpak is similar but community/desktop-focused. **AppImage** is a single portable executable file requiring no installation at all.

Building From Source

```
./configure --prefix=/usr/local       # detect system, generate Makefile
make                                      # compile
sudo make install                           # install the compiled binaries
```

Requires a `build-essential` (Debian) / `Development Tools` (RHEL, via `dnf groupinstall`) toolchain. Downside: the package manager has **no record** of these files, making future updates/removal manual and error-prone — generally avoided in production in favor of proper packages or containers.

Module 15: Storage & Filesystem Management

Block Devices

`/dev/sda` (SCSI/SATA disks), `/dev/nvme0n1` (NVMe SSDs), `/dev/vda` (virtio disks in VMs/cloud). Partitions are numbered suffixes: `/dev/sda1`, `/dev/nvme0n1p1`.

Partitioning Tools

```
fdisk /dev/sdb          # MBR-focused (also supports GPT on modern versions)
# inside fdisk: p=print table, n=new partition, d=delete, t=change type, w=write & exit,
q=quit

gdisk /dev/sdb            # GPT-native equivalent of fdisk

parted /dev/sdb              # works with both MBR and GPT, scriptable
# parted commands: mklabel gpt, mkpart primary ext4 0% 100%, print, rm 1, resizepart
```

Creating Filesystems

```
mkfs.ext4 /dev/sdb1         mkfs.xfs /dev/sdb1        mkfs.btrfs /dev/sdb1
mkfs.vfat /dev/sdb1
mkswap /dev/sdb2              # prepare a partition as swap space
```

Filesystem Repair

```
fsck /dev/sdb1          # generic — dispatches to the right fs-specific checker
e2fsck -f /dev/sdb1       # force check, ext2/3/4
xfs_repair /dev/sdb1
# XFS-specific (note: XFS must be UNMOUNTED to repair, no live fsck)
```

Critical rule: filesystems must normally be **unmounted** before running fsck/repair, or you risk corrupting live data.

Mounting

```
mount /dev/sdb1 /mnt/data
mount -o ro,noexec,nosuid /dev/sdb1 /mnt/data     # read-only, can't execute binaries,
can't honor SUID
umount /mnt/data
umount -l /mnt/data        # lazy unmount — detach now, actually free once no longer busy
mount --bind /src /dst       # bind mount — same content, different path
mount -t tmpfs tmpfs /mnt/ramdisk -o size=512m    # RAM-backed temporary filesystem
blkid                            # show UUIDs + filesystem types for all block devices
```

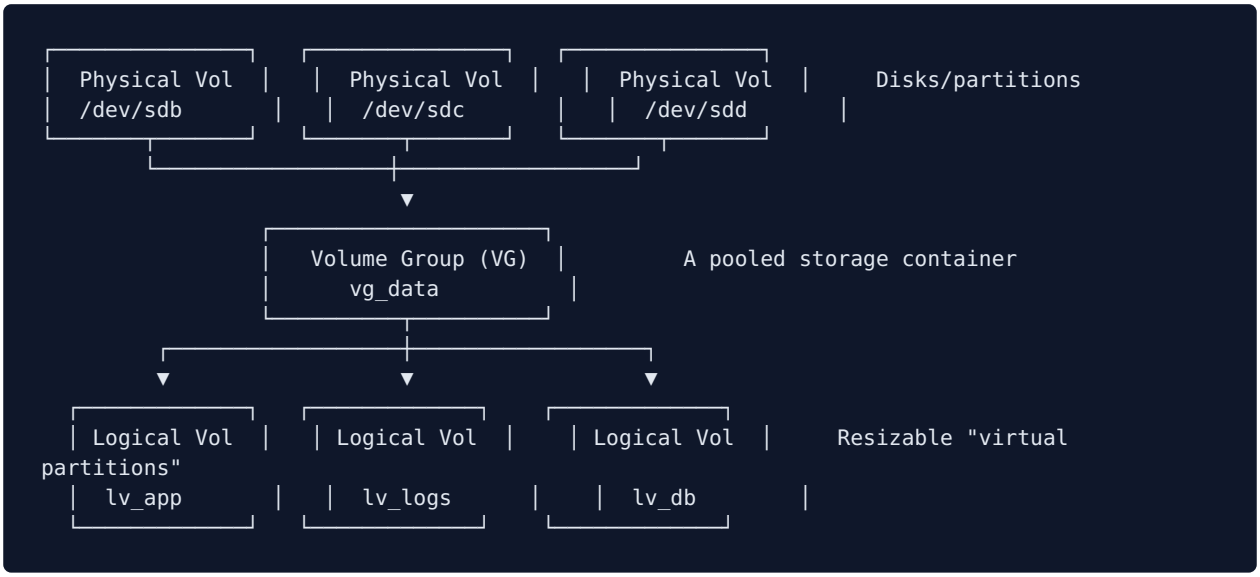
/etc/fstab — Persistent Mounts

```
# device                                mountpoint   fstype  options         dump  pass
UUID=1234-5678-ABCD-EF00               /data         ext4    defaults        0     2
/dev/mapper/vg_data-lv_logs             /var/log      xfs     defaults,noatime 0    2
```

Fields: device (prefer **UUID** over `/dev/sdX` since device names can shift across reboots), mountpoint, filesystem type, mount options, `dump` (legacy backup flag, usually 0), `pass` (fsck order — 1 for root, 2 for others, 0 to skip).

```
e2label /dev/sdb1 mydata         # label an ext filesystem
xfs_admin -L mydata /dev/sdb1      # label an XFS filesystem
mount -a                             # mount everything in fstab (good way to test for
typos safely)
```

LVM — Logical Volume Manager



```
pvcreate /dev/sdb /dev/sdc              pvdisplay           pvs
vgcreate vg_data /dev/sdb /dev/sdc        vgextend vg_data /dev/sdd       vgdisplay    vgs
lvcreate -L 20G -n lv_app vg_data           lvcreate -l 100%FREE -n lv_db vg_data   lvs
lvextend -L +10G /dev/vg_data/lv_app            # grow the logical volume
resize2fs /dev/vg_data/lv_app                      # then grow the ext4 filesystem on top
(xfs_growfs for XFS)
lvremove /dev/vg_data/lv_app
```

LVM types: linear (simple concatenation), striped (performance — like RAID0), mirrored (redundancy), thin (over-provision storage, allocate on demand), cache (SSD as a cache tier for slower disks). **Snapshots**: `lvcreate -L 5G -s -n lv_app_snap /dev/vg_data/lv_app` — captures a point-in-time copy for safe backups or rollback. Extending LVM + the filesystem together is the standard way storage is grown **online**, without downtime — a very common real-world DevOps task.

RAID (mdadm — Software RAID)

```
mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc   # mirror
cat /proc/mdstat        # live status of all software RAID arrays
mdadm --detail /dev/md0    # detailed array info
```

RAID Level	Description
0	Striping — speed, zero redundancy
1	Mirroring — full redundancy, 50% capacity
5	Striping + 1 parity disk — tolerates 1 disk failure
6	Striping + 2 parity disks — tolerates 2 disk failures
10	Mirrored stripes — speed + redundancy

Disk Quotas

```
# mount with usrquota,grpquota in fstab, then:
quotacheck -cug /home       quotaon /home         quotaoff /home
edquota -u shaam               # interactively edit a user's quota
repquota /home                    # report current usage vs quota
setquota -u shaam 5G 6G 0 0 /home   # soft=5G hard=6G blocks
```

Swap Management

```
swapon /dev/sdb2         swapoff /dev/sdb2        swapon --show
mkswap /dev/sdb2            # prepare a partition
dd if=/dev/zero of=/swapfile bs=1M count=2048 && chmod 600 /swapfile && mkswap /swapfile
&& swapon /swapfile
sysctl vm.swappiness=10        # lower = prefer RAM over swap (default is often 60)
```

Storage Performance Tools

```
iostat -x 2                  # extended I/O stats, refresh every 2s
iotop                           # top-style view, per-PROCESS disk I/O
fio --name=test --rw=randwrite --bs=4k --size=1G --numjobs=4    # flexible I/O benchmark
dd if=/dev/zero of=testfile bs=1M count=1024 oflag=direct   # raw sequential write test
hdparm -I /dev/sda               # disk parameters/capabilities
nvme list                          # NVMe device management (nvme-cli)
```

Advanced Filesystems

- **Btrfs**: copy-on-write filesystem with built-in **snapshots**, **subvolumes** (independently mountable sub-trees), native **RAID**, transparent compression, and deduplication.
- **ZFS on Linux**: enterprise-grade checksummed filesystem + volume manager, strong data-integrity guarantees, used heavily in storage appliances.
- **tmpfs**: memory-backed filesystem — blazing fast, but contents vanish on unmount/reboot.
- **overlayfs**: a union filesystem that layers a read-only lower directory under a writable upper directory — this is the exact mechanism Docker uses to make container images "layered" (see Module 22).

Module 16: Networking in Linux

Interface & Address Management (the `ip` command — modern replacement for ifconfig/route/arp)

```
ip addr show                    ip addr add 192.168.1.50/24 dev eth0
ip link show                      ip link set eth0 up          ip link set eth0 down
ip route show                       ip route add default via 192.168.1.1
ip neigh show                         # ARP/neighbor table
ip rule show                            # policy-based routing rules
ip netns add ns1                          # create a network namespace
ip tunnel show                              # tunnel interfaces (GRE, etc.)
ifconfig eth0                                  # legacy — still seen on older systems
```

NetworkManager

```
nmcli device status              nmcli connection show
nmcli connection up "Wired connection 1"
nmcli device wifi connect "SSID" password "secret"
nmtui                              # text-based UI, great over SSH
```

Config locations vary by distro: `/etc/network/interfaces` (Debian legacy), `/etc/sysconfig/network-scripts/` (RHEL legacy), `/etc/NetworkManager/system-connections/` (modern NM), `/etc/systemd/network/` (systemd-networkd).

DNS Configuration

```
/etc/resolv.conf       → nameserver 8.8.8.8  (often auto-managed by systemd-resolved/
NetworkManager now)
/etc/hosts              → static hostname-to-IP overrides (127.0.0.1 localhost, etc.)
/etc/nsswitch.conf        → order of lookup sources (files before dns, etc.)
```

Hostname

```
hostname                   hostnamectl set-hostname web01.internal.local
cat /etc/hostname
```

Network Troubleshooting Toolkit

```
ping -c 4 google.com                    ping6 -c 4 ::1
traceroute google.com                     tracepath google.com
mtr google.com                               # live combined ping + traceroute

ss -tuln                    # all listening TCP/UDP ports (modern replacement for netstat)
ss -tunp                      # same + owning process
ss -s                            # summary statistics
netstat -tulpn                     # legacy equivalent, still very common

tcpdump -i eth0 host 10.0.0.5 and port 443       # filtered packet capture
tcpdump -i eth0 -w capture.pcap                     # write to file for later analysis
(e.g., in Wireshark)
tcpdump -r capture.pcap                               # read a saved capture

nmap -sV 192.168.1.0/24          # service version detection across a subnet
nmap -p 1-1000 -A target.com        # aggressive scan of port range (incl. OS detection)

curl -v https://api.example.com          curl -I https://example.com     # headers only
curl -L -o file.zip https://example.com/file.zip      # follow redirects, save to file
curl -X POST -H "Content-Type: application/json" -d '{"key":"val"}' https://api/endpoint
curl -u user:pass https://api/secure            curl -k https://self-signed.local

wget https://example.com/file.tar.gz

dig example.com                 dig @8.8.8.8 example.com MX      dig +short example.com
dig +trace example.com             # show the full resolution path from root servers
nslookup example.com

nc -zv example.com 443           # quick port-open test
nc -l 8080                          # listen on a port
nc -lvp 9000 > received_file          # receive a file transfer

iperf3 -s                      # server mode
iperf3 -c 10.0.0.5               # client mode — measure throughput between two hosts

ethtool eth0                    # NIC speed/duplex/link settings
arp -a                             # ARP cache (legacy; `ip neigh` is modern equivalent)
```

Firewall Management

iptables (legacy but still everywhere):

```
iptables -L -n -v                          # list current rules, numeric, verbose
iptables -A INPUT -p tcp --dport 22 -j ACCEPT   # append rule to INPUT chain
iptables -I INPUT 1 -s 10.0.0.0/8 -j DROP          # insert at position 1
iptables -D INPUT 3                                   # delete rule #3
iptables -F                                             # flush all rules (careful!)
iptables-save > /etc/iptables/rules.v4                    # persist rules
iptables-restore < /etc/iptables/rules.v4
```

Tables: filter (default — allow/block), nat (address translation), mangle (packet alteration), raw, security. **Chains**: INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING. **Targets**: ACCEPT, DROP, REJECT, LOG, MASQUERADE, DNAT, SNAT.

nftables (modern replacement):

```
nft list ruleset
nft add table inet filter
nft add chain inet filter input { type filter hook input priority 0 \; }
nft add rule inet filter input tcp dport 22 accept
```

firewalld (RHEL/CentOS management layer over nftables/iptables):

```
firewall-cmd --state                      firewall-cmd --list-all
firewall-cmd --zone=public --add-service=http --permanent
firewall-cmd --zone=public --add-port=8080/tcp --permanent
firewall-cmd --reload                          # apply permanent rules
firewall-cmd --get-zones                          # public, internal, dmz, trusted, home,
work...
firewall-cmd --add-rich-rule='rule family="ipv4" source address="10.0.0.5" accept'
```

ufw (Ubuntu's simple frontend):

```
ufw enable               ufw status verbose
ufw allow 22/tcp             ufw deny 23           ufw limit ssh      ufw app list
```

Bonding, Bridging, VLANs

```
# Bonding modes: active-backup, balance-rr, balance-xor, broadcast, 802.3ad (LACP),
balance-tlb, balance-alb
cat /etc/modprobe.d/bonding.conf

ip link add br0 type bridge           # modern bridge creation
brctl show                              # legacy bridge tool

ip link add link eth0 name eth0.100 type vlan id 100    # 802.1Q VLAN tagging
```

Traffic Control & Namespaces

```
tc qdisc add dev eth0 root netem delay 100ms     # simulate 100ms latency (great for chaos
testing)
ip netns add testns                ip netns exec testns ip addr
ip link add veth0 type veth peer name veth1          # virtual ethernet pair — connects
namespaces
```

Network namespaces give each container/VM its own isolated network stack (interfaces, routes, iptables rules) — this is the exact kernel primitive Docker and Kubernetes pods are built on (Module 26).

Module 17: Linux Security

sudo Hardening

Restrict to specific commands (never blanket `ALL=(ALL) ALL` in production), enable logging (`Defaults logfile=/var/log/sudo.log`), consider `Defaults requiretty`, and tune the credential cache timeout (`Defaults timestamp_timeout=5`).

SSH Hardening (`/etc/ssh/sshd_config`)

```
PermitRootLogin no                # never allow direct root SSH login
PasswordAuthentication no          # force key-based auth only
AllowUsers shaam deploy              # whitelist specific accounts
MaxAuthTries 3                         # limit brute-force attempts per connection
LoginGraceTime 30                        # disconnect unauthenticated sessions quickly
ClientAliveInterval 300                    # keepalive ping interval
ClientAliveCountMax 2                        # disconnect after missed keepalives
```

```
systemctl reload sshd           # apply changes without dropping active sessions
ssh-keygen -t ed25519 -C "shaam@laptop"    # generate a modern key pair
ssh -J jumphost target_host                  # ProxyJump — hop through a bastion host
```

Other SSH hardening topics: `authorized_keys` management (one key per line in `~/.ssh/authorized_keys`), `ssh-agent` for key caching, agent forwarding (`-A`, security trade-off), SFTP **chroot jails** (`ChrootDirectory`) to confine file-transfer-only users, `~/.ssh/config` for per-host connection shortcuts, `~/.ssh/known_hosts` for host-key pinning, and **SSH Certificate Authorities** for scalable key management across large fleets (no more distributing individual public keys).

PAM & Password Policy

```
/etc/security/pwquality.conf    → minlen=12, dcredit=-1, ucredit=-1  (complexity
requirements)
/etc/login.defs                  → PASS_MAX_DAYS, PASS_MIN_DAYS, PASS_WARN_AGE
```

`pam_faillock` locks accounts after N failed attempts; `pam_pwquality` enforces password strength at creation time; `pam_access` restricts login by host/user/time combinations; `pam_time` restricts login to specific time windows.

Auditing Files for Risk

```
find / -xdev -perm -0002 -type f 2>/dev/null    # world-writable files
find / -perm -4000 -o -perm -2000 2>/dev/null     # SUID/SGID files
chattr +i /etc/important.conf                        # make critical configs immutable
```

SELinux (Security-Enhanced Linux) — Mandatory Access Control

Unlike standard Unix permissions (**discretionary** — owner decides), SELinux enforces **mandatory** policy-based rules that confine even root.

```
getenforce                  setenforce 0           setenforce 1       # permissive /
enforcing
cat /etc/selinux/config       # persistent default (SELINUX=enforcing|permissive|disabled)
ls -Z /var/www/html              # view file contexts (user:role:type:level)
ps -Z                               # view process contexts
chcon -t httpd_sys_content_t file.html     # TEMPORARY context change
restorecon -Rv /var/www/html                 # restore to the policy-defined default
context
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"   # PERMANENT context rule
semanage port -a -t http_port_t -p tcp 8081      # label a non-standard port for a service
getsebool -a | grep httpd                          # list booleans (toggles) for a service
setsebool -P httpd_can_network_connect on             # persistent boolean change
audit2allow -a                                           # generate a custom policy FROM
denial logs
audit2why -a                                                # explain WHY something was
denied
cat /var/log/audit/audit.log | grep denied                  # raw denial events
```

SELinux troubleshooting workflow: see a mysterious “Permission denied” despite correct Unix permissions → check `getenforce` → check `/var/log/audit/audit.log` for `denied` entries → use `audit2why` to understand → fix via `restorecon` (if context just drifted) or `semanage` / `setsebool` (if policy genuinely needs updating) — avoid just disabling SELinux as a shortcut in production.

AppArmor (Ubuntu/Debian’s MAC alternative)

```
aa-status               aa-enforce /etc/apparmor.d/usr.sbin.nginx       aa-complain /etc/
apparmor.d/usr.sbin.nginx
```

AppArmor is path-based (profiles attached to binaries by path) versus SELinux’s label-based model — generally considered simpler to write but slightly less granular.

Linux Audit Framework

```
systemctl status auditd
auditctl -w /etc/passwd -p wa -k passwd_changes     # watch a file for writes/attribute
changes
ausearch -k passwd_changes                             # search by the tag ("key") you set
aureport --summary                                        # high-level report
```

Kernel-Level Security Features

ASLR (randomizes memory layout to thwart exploit chaining), **NX/XD bit** (marks memory non-executable to block code injection), **stack canaries** (detect buffer-overflow stack corruption before a function returns), **seccomp** (restricts which syscalls a process may even make — core to container security), **capabilities** and **namespaces** (covered in Modules 11 and 22).

Security Scanning Tools

```
lynis audit system        # broad security audit with a hardening score
oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis --results results.xml
ssg-rhel8-ds.xml  # OpenSCAP + CIS benchmark
rkhunter --check              chkrootkit        # rootkit detection
```

Cryptography Tools

```
openssl genrsa -out key.pem 4096                    openssl req -new -key key.pem -out
csr.pem
openssl x509 -in cert.pem -text -noout                 openssl s_client -connect
example.com:443
openssl enc -aes-256-cbc -in file -out file.enc            openssl dgst -sha256 file

gpg --gen-key               gpg -e -r recipient file          gpg -d file.gpg
gpg --sign file                 gpg --verify file.sig
gpg --import key.asc               gpg --export --armor > mykey.asc

ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -C "comment"       # Ed25519 preferred over RSA
for new keys

cryptsetup luksFormat /dev/sdb1          cryptsetup open /dev/sdb1 secure_vol      # LUKS
full-disk encryption
cat /etc/crypttab                           # persistent LUKS mappings at boot
```

Module 18: Logging & Monitoring

Syslog Facilities & Severities

Facilities: kern, user, mail, daemon, auth, syslog, lpr, news, uucp, cron, local0-7. **Severities** (most to least severe): emerg, alert, crit, err, warning, notice, info, debug.

rsyslog

```
/etc/rsyslog.conf and /etc/rsyslog.d/
*.info;mail.none;authpriv.none;cron.none    /var/log/messages      # facility.severity
action
auth,authpriv.*                              /var/log/secure
*.*   @@logserver.internal:514                   # remote forwarding (TCP, via @@)
```

journald

```
# /etc/systemd/journald.conf
Storage=persistent        SystemMaxUse=500M
```

logrotate

```
# /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    postrotate
        systemctl reload myapp > /dev/null 2>&1 || true
    endscript
}
```

```
logrotate -d /etc/logrotate.d/myapp     # dry run (debug) — shows what WOULD happen
logrotate -f /etc/logrotate.d/myapp       # force an immediate rotation
```

Important Log Files

Path	Content
`/var/log/messages` (RHEL) / `/var/log/syslog` (Debian)	General system log
`/var/log/auth.log` / `/var/log/secure`	Authentication events
`/var/log/kern.log`	Kernel messages
`/var/log/dmesg`	Boot-time kernel ring buffer snapshot
`/var/log/cron`	Cron job execution
`/var/log/audit/audit.log`	SELinux/audit subsystem
`/var/log/wtmp`, `/var/log/btmp`	Successful / failed login records (binary, read via `last` / `lastb`)

```
dmesg -T              # human-readable timestamps
dmesg -w                # follow live (like tail -f, for kernel ring buffer)
dmesg -l err,crit         # filter by severity
```

System Monitoring Tools

```
vmstat 2 5            # 5 samples, 2s apart — procs, memory, swap, io, cpu
iostat -x 2             # extended CPU + per-device I/O stats
mpstat -P ALL 2            # per-CPU-core statistics
sar -u 2 5
# CPU usage over time (requires sysstat package + /var/log/sa/ history)
free -h                        # memory: total/used/free/shared/buff-cache/available
uptime                            # load average: 1, 5, 15-minute windows
lscpu                                lsmem          lsblk          lspci
lsusb         lshw
dmidecode -t system                    # BIOS/hardware inventory
cat /proc/cpuinfo /proc/meminfo /proc/loadavg /proc/uptime /proc/net/dev
```

Load average interpretation: a value of `4.0` on a 4-core machine means the CPUs are, on average, exactly fully utilized; above that, processes are queued waiting for CPU time. Load average reflects *demand* (including processes waiting on I/O in state D), not pure CPU %.

Performance Analysis Methodology

- **USE Method** (Brendan Gregg): for every resource, check **U**tilization, **S**aturation, **E**rrors.
- **RED Method**: for services, check **R**ate, **E**rrors, **D**uration — the standard framing behind most Prometheus/Grafana service dashboards.

```
perf stat ls              perf record -g ./app      perf report      perf top
bpftrace -e 'tracepoint:syscalls:sys_enter_open { printf("%s\n", comm); }'   # eBPF one-
liner
```

Flame graphs visualize CPU profiling data as a stack of function calls, width = time spent — the fastest way to spot a hot function in a complex app.

Module 19: Scheduling & Automation

cron

```
crontab -e                # edit current user's crontab
crontab -l                  # list it
crontab -r                    # remove it entirely
```

```
# minute hour day month weekday   command
  0      2    *   *     *          /opt/scripts/backup.sh
  */15   *    *   *     *          /opt/scripts/healthcheck.sh
  0      0    1   *     *          /opt/scripts/monthly_report.sh
@reboot  /opt/scripts/startup.sh
@daily   /opt/scripts/cleanup.sh
```

System-wide jobs also live in `/etc/crontab` and `/etc/cron.d/`, plus the convenience directories `/etc/cron.{hourly,daily,weekly,monthly}/`. `/etc/cron.allow` and `/etc/cron.deny` control who may schedule jobs. **Common pitfall**: cron runs with a minimal `PATH` and no interactive shell environment — always use absolute paths inside cron scripts.

anacron

Designed for machines that aren't always on (e.g., laptops) — runs missed daily/weekly/monthly jobs once the system is back up, configured in `/etc/anacrontab`.

at / batch (one-time scheduling)

```
at 14:30                # drop into a prompt, type commands, Ctrl+D to save
atq                       # list pending at jobs
atrm 3                      # cancel job #3
at -f script.sh now + 1 hour     # schedule a script to run in 1 hour
batch                              # run when system load permits
```

`/etc/at.allow` / `/etc/at.deny` mirror the cron access-control pattern.

Automation Patterns

Production automation scripts should be **idempotent** (safe to re-run without side effects), use **lock files** (`flock`) to prevent two copies running concurrently, log clearly with timestamps, and alert (email/Slack/webhook) on failure rather than failing silently.

Module 20: Network Services Administration

SSH Server (OpenSSH)

Already covered in depth in Module 17 (hardening). Additional admin notes: host keys live in `/etc/ssh/ssh_host_*_key`; the SFTP subsystem is configured via the `Subsystem sftp` directive in `sshd_config`; SSH tunneling (`ssh -L`, `-R`, `-D`) forwards arbitrary TCP traffic through an encrypted channel.

Web Servers

Apache HTTPD

```
/etc/httpd/conf/httpd.conf              # main config
<VirtualHost *:80>
    ServerName example.com
    DocumentRoot /var/www/example
</VirtualHost>
```

```
apachectl configtest        apachectl graceful       systemctl status httpd
```

Key modules: `mod_ssl` (TLS), `mod_rewrite` (URL rewriting), `mod_proxy` (reverse proxying), `mod_auth` (authentication). `.htaccess` allows per-directory overrides without touching the main config.

Nginx

```
/etc/nginx/nginx.conf
/etc/nginx/conf.d/example.conf
server {
    listen 80;
    server_name example.com;
    location /api/ {
        proxy_pass http://127.0.0.1:8080;
    }
}
```

```
nginx -t            # test config syntax BEFORE reloading — always do this first
nginx -s reload        # graceful reload, zero dropped connections
```

Nginx is frequently used as a **reverse proxy** and **load balancer** in front of application servers (Node, Gunicorn, etc.).

Let’s Encrypt / Certbot

```
certbot --nginx -d example.com           # obtain + auto-configure Nginx
certbot renew --dry-run                    # test auto-renewal (a systemd timer handles
the real renewals)
```

DNS Server (BIND)

```
/etc/named.conf                 # main config: zones, options, ACLs
/var/named/example.com.zone       # zone file: A, AAAA, MX, CNAME, NS, TXT records
named-checkconf                     named-checkzone example.com /var/named/
example.com.zone
```

Roles: **authoritative** (owns the zone data), **recursive** (resolves queries on behalf of clients), **caching** (speeds up repeated lookups).

DHCP Server

```
/etc/dhcp/dhcpd.conf
subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200;
    option routers 192.168.1.1;
}
host server1 { hardware ethernet 08:00:27:aa:bb:cc; fixed-address 192.168.1.10; }
```

Active leases are tracked in `/var/lib/dhcp/dhcpd.leases`.

NFS (Network File System)

```
/etc/exports
/data  10.0.0.0/24(rw,sync,no_root_squash)
```

```
exportfs -ra            # reload exports
mount -t nfs server:/data /mnt/data
```

`root_squash` (default — remote root is mapped to an unprivileged user, a key security control), `no_root_squash` (disables that protection — use with care), `all_squash` (maps ALL remote users to one unprivileged account). NFSv4 adds built-in ACL support and optional Kerberos-based security (`sec=krb5`).

Samba (Windows File Sharing)

```
/etc/samba/smb.conf
[shared]
   path = /srv/shared
   browsable = yes
   writable = yes
```

```
smbpasswd -a shaam        testparm         systemctl status smb nmb
```

FTP Server (vsftpd)

Config at `/etc/vsftpd/vsftpd.conf`; key decisions are anonymous vs. authenticated access, passive-mode port ranges (needed to work through firewalls/NAT), and enabling TLS (`ssl_enable=YES`) since plain FTP sends credentials unencrypted.

Email (Postfix)

```
/etc/postfix/main.cf        → myhostname, mydestination, relayhost
```

```
postfix check             postfix reload
mailq                       postqueue -f         postsuper -d ALL    # manage the mail
queue
```

Time Sync (Chrony — modern default on RHEL 8+)

```
/etc/chrony.conf     → server pool.ntp.org iburst
```

```
chronyc sources           chronyc tracking         chronyc makestep
timedatectl set-ntp true
```

Proxy Server (Squid)

Basic config at `/etc/squid/squid.conf`; access is controlled via ACLs (`acl localnet src 10.0.0.0/8`, then `http_access allow localnet`). Can run as a **transparent proxy**, intercepting traffic without client-side configuration.

PART 4: ADVANCED LINUX

Module 21: Kernel Management

Kernel Versioning

Kernel versions follow `major.minor.patch` (e.g., `6.6.10`). You can check yours with `uname -r`.

Kernel Modules

Modules are pluggable drivers/features loaded on demand rather than compiled permanently into the kernel.

```
lsmod                          # list currently loaded modules
modinfo e1000e                   # details about a module (description, params, license)
modprobe e1000e                    # load a module AND its dependencies
modprobe -r e1000e                   # unload (and dependents if unused)
insmod /path/module.ko                  # low-level load — no dependency resolution
rmmod e1000e                              # low-level unload
```

Persistent configuration: `/etc/modules-load.d/*.conf` (load at boot), `/etc/modprobe.d/*.conf` (module parameters/options), `/etc/modprobe.d/blacklist.conf` (prevent a module from auto-loading — common for disabling an unwanted driver).

Kernel Parameters (sysctl)

```
sysctl -a | less                      # view ALL current runtime kernel parameters
sysctl net.ipv4.ip_forward               # view a single value
sysctl -w net.ipv4.ip_forward=1            # change NOW (lost on reboot unless persisted)
echo 1 > /proc/sys/net/ipv4/ip_forward       # equivalent low-level method
```

Persist changes in `/etc/sysctl.conf` or `/etc/sysctl.d/99-custom.conf`, then `sysctl -p` to apply.

Parameter	Purpose
`net.ipv4.ip_forward`	Enable routing between interfaces (needed for NAT gateways, Docker networking)
`net.ipv4.tcp_syncookies`	SYN flood protection
`vm.swappiness`	How aggressively the kernel swaps (0-100)
`vm.dirty_ratio`	% of RAM that can hold unwritten (dirty) pages before forced flush
`fs.file-max`	System-wide max open file descriptors
`kernel.pid_max`	Maximum PID value — matters on large systems/Kubernetes nodes
`net.core.somaxconn`	Max queued connections per listening socket
`net.ipv4.tcp_max_syn_backlog`	Max queued half-open TCP connections
`kernel.shmmax`	Max single shared-memory segment size
`net.ipv4.ip_local_port_range`	Ephemeral port range for outgoing connections

Kernel Updates & Crash Dumps

```
dnf install kernel          # installs new kernel ALONGSIDE existing ones (safe rollback
via GRUB menu)
grub2-set-default 0           # choose which kernel boots by default
package-cleanup --oldkernels --count=2   # remove old kernels, keep 2 newest
systemctl status kdump           # crash-dump capture service
cat /etc/kdump.conf                 # where dumps are saved, which kernel to boot into for
dumping
crash /var/crash/vmcore vmlinux       # analyze a captured crash dump
```

eBPF (extended Berkeley Packet Filter)

eBPF lets you run small, **verified-safe** sandboxed programs inside the kernel — without writing kernel modules or recompiling the kernel — to observe or modify behavior at near-zero overhead. The kernel's **verifier** statically proves the program can't crash the kernel or loop forever before it's allowed to load.

```
bpftrace -l 'tracepoint:syscalls:*open*'      # list available tracepoints
bpftrace -e 'kprobe:do_sys_open { printf("%s %s\n", comm, str(arg1)); }'
execsnoop                tcpconnect              biolatency              runqlat   #
common BCC tools
```

Use cases: deep observability (Prometheus-style metrics with microsecond precision), security enforcement (Cilium, Falco), and advanced networking (Cilium's eBPF-based CNI for Kubernetes).

Module 22: Linux Containers & Virtualization

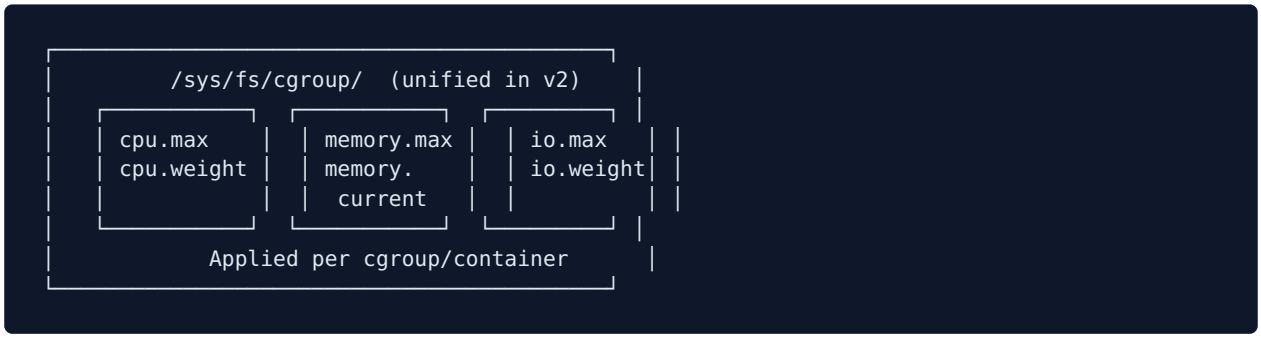
Namespaces — The Isolation Primitive

Namespaces make a process believe it has its own private view of a global kernel resource.

Namespace	Isolates
`pid`	Process IDs — container sees itself as PID 1
`net`	Network interfaces, routes, ports
`mnt`	Filesystem mount points
`uts`	Hostname & domain name
`ipc`	Inter-process communication (shared memory, semaphores)
`user`	UID/GID mapping (root in container ≠ root on host)
`cgroup`	Visibility of the cgroup hierarchy itself
`time`	System clock (Linux 5.6+)

```
unshare --pid --net --mount --fork bash
# manually create new namespaces and drop into a shell
nsenter -t 1234 -n -m                       # ENTER the namespaces of an already-running
process (PID 1234)
lsns                                           # list all active namespaces on the system
```

cgroups (Control Groups) — The Resource-Limiting Primitive



cgroups v1 had separate hierarchies per subsystem (cpu, memory, blkio…); **cgroups v2** unifies everything into a single hierarchy, which is what modern Docker/Kubernetes nodes use.

```
cat /sys/fs/cgroup/memory/docker/<id>/memory.limit_in_bytes      # v1 example
cat /sys/fs/cgroup/<container>/memory.max                           # v2 example
systemd-cgls             # visualize the whole cgroup tree
```

systemd itself manages cgroups for every service it runs — this is how `systemctl` can enforce `MemoryMax=` / `CPUQuota=` directives per service.

Overlay Filesystem — How Container Images Are Layered

```
merged/   ◄── what the container actually sees (read+write)
  ▲
upper/    ◄── writable layer — ALL changes the container makes go here
  ▲
lower/    ◄── read-only image layers, stacked (base image + each Dockerfile RUN layer)
  ▲
work/     ◄── internal bookkeeping directory (required, not directly used)
```

This is exactly why containers are fast to start (no copying) and why deleting a container discards only the thin writable `upper` layer.

KVM (Kernel-based Virtual Machine)

```
lsmod | grep kvm             # confirm hardware virtualization (VT-x/AMD-V) support is
active
virsh list --all               virsh start myvm        virsh shutdown myvm       virsh
destroy myvm
virsh define myvm.xml             virsh undefine myvm
virt-install --name vm1 --memory 2048 --vcpus 2 --disk size=20 --cdrom ubuntu.iso
virt-manager                         # GUI front-end to libvirt
```

QEMU is the actual userspace hardware emulator; KVM is the kernel module giving it direct hardware-accelerated CPU access; **libvirt** is the management API/daemon that `virsh` / `virt-manager` talk to. VM disks commonly use `qcow2` (supports snapshots, thin provisioning) vs `raw` (simpler, slightly faster). Networking modes: NAT (outbound only, simplest), bridged (VM gets its own LAN-visible IP), isolated (VM-to-VM only).

LXC/LXD — System Containers

```
lxc launch ubuntu:22.04 mycontainer      lxc list       lxc exec mycontainer
-- bash       lxc stop mycontainer
```

LXC containers run a **full OS userspace** (like a lightweight VM) rather than a single application process — this is the key distinction from Docker, which is designed around single-process, single-purpose containers.

Docker on Linux — Kernel-Level View

Docker is largely "glue" on top of the primitives above: `dockerd` / `containerd` manage lifecycle, `runc` (an OCI-spec-compliant runtime) actually calls into the kernel to set up namespaces + cgroups + overlayfs for each container, and Docker's networking creates a `docker0` bridge plus `iptables` NAT rules so containers can reach the outside world.

Podman — Daemonless Alternative

```
podman run -d --name web nginx       podman ps       podman-compose up
```

Podman has **no central daemon** (each container is a direct child process of the user who started it) and natively supports **rootless containers** — running containers without any root privileges at all, using user namespaces for UID remapping.

systemd-nspawn

A lightweight, systemd-native containerization tool — useful for quick chroot-like isolated environments without pulling in a full container runtime.

Module 23: High Availability & Clustering

Pacemaker + Corosync

```
pcs status                pcs resource create webserver ocf:heartbeat:IPaddr2
ip=10.0.0.100
pcs constraint colocation add webserver with postgresql
pcs property set stonith-enabled=true
```

Corosync handles cluster membership/communication and **quorum** (preventing split-brain by requiring a majority of nodes to agree before acting). **Pacemaker** is the resource manager sitting on top, deciding which node runs which service and failing it over automatically. **STONITH** ("Shoot The Other Node In The Head") forcibly powers off/fences a misbehaving node to guarantee it can't corrupt shared data.

Keepalived (VRRP-based failover)

```
# /etc/keepalived/keepalived.conf
vrrp_instance VI_1 {
    state MASTER
    virtual_router_id 51
    priority 150
    virtual_ipaddress { 10.0.0.100 }
}
```

Keepalived is the lightweight way to give two servers a shared **floating/virtual IP** that instantly moves to the backup if the primary's health check fails.

HAProxy — Load Balancing

```
frontend web_front
    bind *:80
    default_backend web_back
backend web_back
    balance roundrobin
    server web1 10.0.0.1:80 check
    server web2 10.0.0.2:80 check
```

Supports health checks, SSL termination, ACL-based routing rules, and a built-in `/stats` dashboard page.

Distributed Storage: GlusterFS & Ceph

- **GlusterFS**: aggregates “bricks” (directories on different servers) into distributed, replicated, or dispersed (erasure-coded) volumes, mounted like any normal filesystem.
- **Ceph**: a full distributed storage platform. Architecture = **MON** (monitors — cluster state/quorum), **MGR** (manager/metrics), **OSD** (Object Storage Daemons — actual data), **MDS** (metadata server, for CephFS only). Exposes three interfaces: **RADOS** (raw object store), **RBD** (block devices), **RGW** (S3-compatible object gateway), and **CephFS** (POSIX filesystem).

```
ceph status          ceph health detail       ceph osd tree
```

Module 24: Performance Tuning

CPU Tuning

```
cpupower frequency-info            cpupower frequency-set -g performance
taskset -c 0,1 ./myapp                 # pin a process to specific CPU cores
cat /proc/irq/24/smp_affinity             # which CPU(s) handle a given interrupt
```

`isolcpus=2,3` (kernel boot parameter) reserves cores exclusively for latency-sensitive workloads, keeping the scheduler from placing other tasks there. CPU governors: `performance` (max clock always), `powersave`, `ondemand` / `schedutil` (dynamic scaling).

Memory Tuning

```
sysctl vm.swappiness=10                     # prefer RAM over swap
echo 20 > /sys/kernel/mm/transparent_hugepage/enabled   # (or 'never'/'always') — THP can
hurt DB latency
sysctl vm.nr_hugepages=1024                   # reserve standard (non-transparent) huge
pages
numactl --hardware                              # view NUMA topology
numactl --cpunodebind=0 --membind=0 ./app          # pin process + its memory to one NUMA
node
sysctl vm.dirty_ratio=20 vm.dirty_background_ratio=10
cat /proc/1234/oom_score_adj                          # adjust OOM-killer preference for a
process (-1000=never kill)
```

Disk I/O Tuning

```
cat /sys/block/sda/queue/scheduler          # current I/O scheduler, options in brackets
echo mq-deadline > /sys/block/sda/queue/scheduler
blockdev --setra 4096 /dev/sda                   # read-ahead size
mount -o noatime,nodiratime /dev/sdb1 /data        # skip access-time updates — real perf
win on busy fs
fstrim -v /                                          # SSD TRIM — reclaim unused blocks
```

I/O schedulers: `mq-deadline` (good general default), `kyber` (low-latency, SSD/NVMe-friendly), `bfq` (fair-queuing for desktop/mixed workloads), `none` (passthrough, best for very fast NVMe where the drive's own controller handles scheduling).

Network Tuning

```
sysctl -w net.core.rmem_max=16777216 net.core.wmem_max=16777216
sysctl -w net.ipv4.tcp_congestion_control=bbr        # Google's BBR — often a big latency/
throughput win
sysctl -w net.core.somaxconn=4096
ethtool -C eth0 rx-usecs 50                            # interrupt coalescing — fewer
interrupts, more latency
ip link set eth0 mtu 9000                                 # jumbo frames (needs switch
support end-to-end)
ethtool -k eth0 | grep tso                                   # check TCP Segmentation
Offload status
```

Application-Level Tuning

```
ulimit -n 65536                     # raise open file descriptor limit (classic "too many
open files" fix)
# /etc/security/limits.conf: myappuser soft nofile 65536 / hard nofile 65536
LD_PRELOAD=/usr/lib/libjemalloc.so ./app
# swap in a different memory allocator without recompiling
```

Benchmarking & Profiling

```
sysbench cpu --threads=4 run                sysbench memory run        sysbench fileio --
file-test-mode=rndrw run
fio --name=randread --rw=randread --bs=4k --size=1G
iperf3 -c target -t 30
stress-ng --cpu 4 --vm 2 --timeout 60s        # synthetic load/stress testing

perf stat ./app                perf record -g ./app        perf report       perf top
valgrind --tool=memcheck ./app       # detect memory leaks/corruption (heavy runtime
overhead)
```

Flame graphs (Brendan Gregg's methodology): capture a `perf record -g`, fold the stacks, render an SVG — instantly shows which function consumes the most CPU time, width-encoded.

Module 25: Linux Hardening & Best Practices

CIS Benchmark Philosophy

The **CIS (Center for Internet Security) Benchmark** for Linux is an industry-standard, itemized checklist (hundreds of controls) covering filesystem config, patching, network config, logging/auditing, and access control — most compliance programs (SOC2, PCI-DSS) map back to it.

Core Hardening Checklist

- **Minimal installation**: install only what's needed — every package is attack surface.
- **Disable unused services**: `systemctl list-unit-files --state=enabled` then disable anything not required.
- **Reduce network exposure**: close every port that isn't explicitly needed (`ss -tuln` to audit).

Kernel Hardening (sysctl)

```
net.ipv4.icmp_echo_ignore_broadcasts = 1    # prevent smurf-style amplification
net.ipv4.conf.all.rp_filter = 1               # reverse-path filtering — drop spoofed-
source packets
net.ipv4.conf.all.accept_source_route = 0       # disallow source-routed packets
net.ipv4.conf.all.accept_redirects = 0            # ignore ICMP redirects (can be used to
hijack routes)
net.ipv4.conf.all.log_martians = 1                   # log suspicious/impossible packets
kernel.dmesg_restrict = 1                              # unprivileged users can't read dmesg
kernel.kptr_restrict = 2                                 # hide kernel pointers from /proc
(anti-exploit)
kernel.randomize_va_space = 2                              # full ASLR
fs.suid_dumpable = 0                                         # don't core-dump SUID
processes (data leak risk)
```

Boot & Filesystem Hardening

```
grub2-setpassword             # require a password to edit GRUB entries at boot
```

Separate partitions for `/tmp`, `/var`, `/home`, `/var/log` so a filled or compromised one can't take down `/`; mount `/tmp` and similar with `noexec,nosuid,nodev`.

Mandatory Access Control & Audit Logging

Already covered in depth in Module 17 — SELinux/AppArmor enforcing policy, `auditd` capturing security-relevant events.

File Integrity Monitoring (AIDE)

```
aide --init                       # build the initial baseline database
mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz
aide --check                        # compare current state vs. baseline — flags any
unexpected changes
```

`/etc/aide.conf` defines which paths/attributes to track. Typically run on a schedule (cron/systemd timer) so unauthorized file changes (e.g., a tampered `/bin/ls`) are caught quickly.

Automated Hardening Tools

```
lynis audit system                          # scores your system + gives specific
remediation tips
ansible-galaxy install dev-sec.hardening      # community Ansible role implementing CIS-
style hardening
```

Many cloud providers also offer **CIS-hardened golden images** so you inherit a hardened baseline instead of hardening from scratch on every new VM.

PART 5: LINUX FOR DEVOPS

Module 26: Linux for Containers & Kubernetes

Namespace Debugging Commands

```
nsenter -t <PID> -n -m -u -i -p bash
# drop into ALL namespaces of a running container's PID
unshare --pid --net --fork bash           # manually create a sandbox namespace
lsns -t net                                 # list all network namespaces on the host
```

`nsenter` is invaluable when `docker exec` / `kubectl exec` isn't available (e.g., a distroless image with no shell) — you jump into the container's namespaces **from the host** instead of needing a shell inside it.

Inspecting Container Resource Limits via cgroups

```
cat /sys/fs/cgroup/memory/docker/<container_id>/memory.limit_in_bytes     # cgroups v1
cat /sys/fs/cgroup/system.slice/docker-<id>.scope/memory.max                # cgroups v2
docker inspect <container> --format '{{.HostConfig.Memory}}'
```

Container Networking at the Kernel Level

```
ip link show                  # look for veth* interfaces — one end of each pair lives
inside a container
brctl show docker0               # or cni0 for Kubernetes CNI bridges
iptables -t nat -L -n | grep DOCKER       # NAT rules Docker injected for port publishing
ipvsadm -L -n                               # kube-proxy IPVS rules (alternative to
iptables mode)
ip netns exec cni-xxxx ip addr                # peek inside a pod's network namespace
directly
```

Overlay Filesystem Inspection

```
ls /var/lib/docker/overlay2/<id>/diff      # see exactly what THIS layer changed
docker inspect <container> --format '{{.GraphDriver.Data}}'   # shows LowerDir/UpperDir/
MergedDir paths
```

Container Debugging Without `exec`

```
nsenter -t $(docker inspect -f '{{.State.Pid}}' mycontainer) -n -m bash
strace -p $(docker inspect -f '{{.State.Pid}}' mycontainer)     # trace syscalls from the
HOST
```

Linux Capabilities in Containers

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE myapp     # principle of least
privilege
cat /proc/<PID>/status | grep Cap                                  # view effective
capabilities (hex-encoded)
```

Docker grants a reduced default capability set (not full root) even to containers running as `root` — dropping further unused capabilities is a core container-hardening practice.

Seccomp Profiles

```
docker run --security-opt seccomp=/path/custom-profile.json myapp
```

Docker's default seccomp profile already blocks ~44 dangerous syscalls (like `reboot`, `mount`); Kubernetes lets you set `seccompProfile` at the Pod or container spec level (`RuntimeDefault` or a custom JSON profile).

Kernel Tuning for Kubernetes Nodes

```
sysctl -w vm.max_map_count=262144                # required by Elasticsearch/OpenSearch
pods
sysctl -w fs.inotify.max_user_watches=524288        # kubelet/file-watchers need many
inotify watches
sysctl -w fs.inotify.max_user_instances=512
sysctl -w net.bridge.bridge-nf-call-iptables=1         # let iptables see bridged traffic
(CNI requirement)
sysctl -w net.ipv4.ip_forward=1                           # required for pod-to-pod
routing
sysctl -w kernel.pid_max=4194304                             # large clusters can hit
default PID ceilings
```

These are typically delivered to nodes via a `DaemonSet` with a privileged init container, or baked into the node's bootstrap/user-data script.

Kubernetes Node-Level Troubleshooting

```
journalctl -u kubelet -f                    # kubelet logs — the most common first stop
journalctl -u containerd -n 200
iptables -L -t nat -n | grep <service-ip>      # debug a Service not routing traffic
nsenter -t <pod_pid> -n -- cat /etc/resolv.conf   # check DNS config actually inside a
pod's netns
nsenter -t <pod_pid> -n -- nslookup kubernetes.default   # debug CoreDNS resolution from
inside a pod
```

Module 27: Linux Automation for DevOps

Git on Linux

```
git config --global user.name "Shaam"           git config --global user.email "you@exampl
e.com"
git config --global init.defaultBranch main
# Hooks — executable scripts dropped into .git/hooks/
#   pre-commit  → lint/format before allowing a commit
#   pre-push    → run tests before allowing a push
#   post-commit   → trigger notifications, etc.
git init --bare /srv/git/myrepo.git                # bare repo — used as a remote/server,
no working tree
git config --global credential.helper store           # cache credentials (or use an SSH
key instead)
```

Build Tools

```
make                     # reads a Makefile — target: dependencies \n\t command
cmake -B build && cmake --build build
mvn clean install            gradle build           npm install && npm run build
pip install -r requirements.txt       cargo build --release        go build ./...
```

A **Makefile** target looks like:

```
deploy: build test
    ansible-playbook deploy.yml
build:
    docker build -t myapp .
test:
    pytest tests/
```

Environment Management

```
python3 -m venv .venv && source .venv/bin/activate      # Python virtual environment
pyenv install 3.12.0           pyenv local 3.12.0           # per-project Python version
nvm install 20                   nvm use 20                   # per-project Node version
asdf install python 3.12.0          asdf local python 3.12.0        # one tool to rule ALL
runtime versions
direnv allow
# auto-load .envrc on `cd` into a project
```

Scripting Cloud & APIs

```
curl -s https://api.github.com/repos/kubernetes/kubernetes | jq '.stargazers_count'
curl -s https://api.example.com/users | jq '.[] | select(.active==true) | .email'
jq '.items[] | {name: .metadata.name, status: .status.phase}' pods.json      # reshape JSON
yq '.spec.replicas' deployment.yaml                                             # YAML
equivalent of jq

aws configure                         aws configure --profile prod
aws ec2 describe-instances --output table
aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId' --output text
aws s3 ls --profile prod --region ap-south-1
aws ec2 describe-instances --output json | jq -r '.Reservations[].Instances[].State.Name'

kubectl get pods -A                   kubectl logs -f deploy/myapp -n prod
kubectl exec -it pod-xyz -- /bin/sh      kubectl apply -f manifest.yaml

terraform init          terraform plan -out=tfplan        terraform apply tfplan
terraform destroy
```

`jq` filter basics: `.key` (field access), `.[0]` (array index), `.[]` (iterate all elements), `|` (pipe filters together), `select()` (filter matching elements), `map()`, `length`, `keys`, `values`.

Cron/Timer-Driven DevOps Tasks

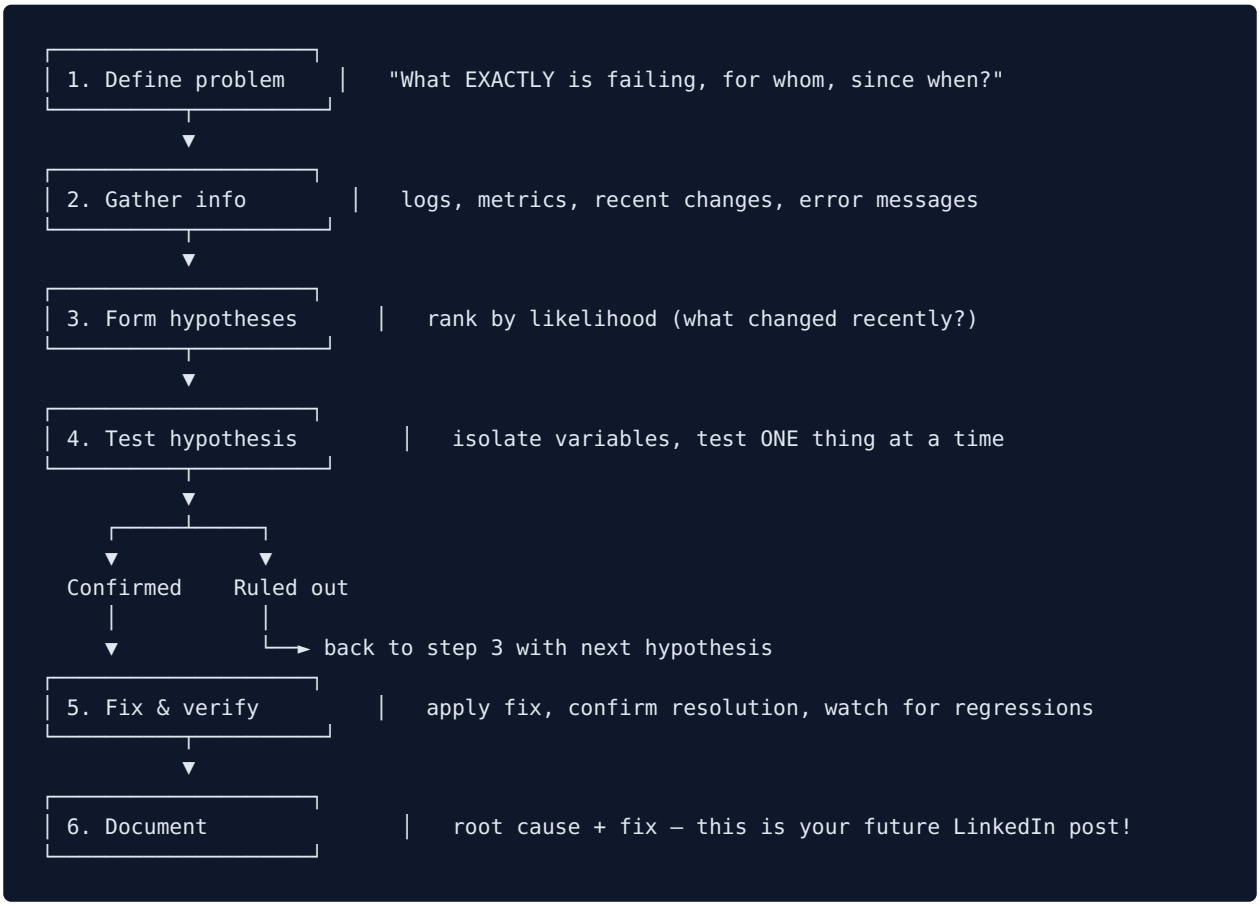
```
0 1 * * *  /opt/scripts/backup_db.sh >> /var/log/backup.log 2>&1
0 3 * * 0  /opt/scripts/cleanup_old_logs.sh
0 */6 * * * /opt/scripts/renew_cert_if_needed.sh
*/5 * * * * /opt/scripts/healthcheck.sh || curl -X POST https://hooks.slack.com/... -d '{"t
ext":"health check failed"}'
```

Linux in CI/CD

CI runners (Jenkins agents, GitLab Runners, GitHub Actions runners) are almost always ephemeral Linux containers/VMs. Key Linux knowledge that matters here: environment variable scoping per job/stage, Docker-in-Docker vs. Docker socket mounting for container-building pipelines, correctly setting `WORKDIR`/artifact paths, and managing file permissions on shared build caches.

Module 28: Linux Troubleshooting Methodology

Structured Troubleshooting Flow



First Commands When Something Is Wrong

```
uname -a               hostnamectl         lscpu          free -h        df -h
rpm -qa --last | head
# (or) grep " install " /var/log/dpkg.log | tail   # recent installs
last -x | head                     git log -5 --oneline          # recent human changes
journalctl -xe                       dmesg -T | tail -50              tail -f /var/log/
messages
```

Common Problem Playbooks

System won’t boot

```
GRUB rescue mode → boot a live ISO → chroot in → grub2-mkconfig / grub2-install
Dropped to emergency target → check `journalctl -xb` → often a bad /etc/fstab entry or
failed fsck
```

Cannot log in Check PAM config (`/etc/pam.d/`), password expiry (`chage -l user`), `/etc/securetty` restrictions, home directory existing with correct ownership, and the user's shell actually existing in `/etc/shells`.

Disk full

```
df -h                                  # which filesystem is full
du -xh --max-depth=2 / | sort -rh | head -20    # biggest directories
lsof +L1                                  # files deleted but still held open by a process
(common log-growth trap)
```

High CPU

```
top -o %CPU                pidstat 2 5         strace -c -p <pid>         perf top
```

Watch for zombie processes (`ps aux | grep Z`) accumulating and starving the process table, and kernel-thread CPU usage (names in brackets, e.g. `[kswapd0]`) hinting at a memory-pressure problem, not an app problem.

High memory usage

```
free -h                ps aux --sort=-%mem | head -10         cat /proc/meminfo
dmesg | grep -i "killed process"      # confirms if the OOM killer fired, and on what
```

High disk I/O

```
iostat -x 2             iotop -o           blktrace -d /dev/sda -o trace
```

Distinguish a genuine storage bottleneck (device `%util` near 100%, high `await`) from an application issue (e.g., a runaway query generating excessive reads).

Network connectivity issues (bottom-up OSI approach)

```
ip link                    # 1. Is the interface even up?
ip addr                     # 2. Does it have the right IP?
ip route                      # 3. Is there a route to the destination?
dig example.com                 # 4. Does DNS resolve?
iptables -L -n                    # 5. Is a firewall rule blocking it?
ss -tuln                             # 6. Is the port actually listening?
ping / traceroute target                # 7. Is the remote host even reachable?
ss -tuln | grep <port>                     # 8. Is YOUR app actually bound and listening?
```

Service won’t start

```
systemctl status myapp                  journalctl -u myapp -n 100
ls -la /path/to/binary                     # permission issues?
ss -tuln | grep <port>                        # port already in use by something else?
getenforce && ausearch -m avc -ts recent          # SELinux denial?
```

Permission denied errors

```
ls -la file                  id username            getfacl file
ausearch -m avc -ts recent --raw | audit2why          # is SELinux the REAL blocker, not
Unix perms?
getcap /path/to/binary                                  # missing a needed capability?
```

Package manager issues

```
dnf history                dnf history undo <id>        rpm -Va | grep '^..5'     # check
for corruption
rm -f /var/lib/rpm/.rpm.lock /var/lib/dpkg/lock*            # stale lock files (use with
caution)
apt-get check                   apt-get install -f                # fix broken
dependencies
```